MOVIEON

Functions and SDLC documentation

Version 1.1.1 · 2026-10-20
Movies · series · anime — free catalog and Silver / Gold / Diamond plans

Contents: (0) functions overview · (1) planning & requirements · (2) system design · (3) testing · (4–6) deployment, operations & maintenance.

Status of this release: server tests 66/66, browser end-to-end 17/17. **Not verified:** the Docker image build (no Docker daemon in the build environment), real payments (demo provider only), and a live VK embed (tested against a stub).

Contents

MovieOn — Functions Overview

1. Who uses MovieOn
2. Plans and what they unlock
3. Viewer-facing functions
4. Admin functions
5. API functions (summary)
6. System functions (no UI)
7. Known boundaries (read before launching)

MovieOn — Phase 1: Planning & Requirements

1. Vision
2. Content and legal model (critical constraint)
3. SDLC model
4. Stakeholders and personas
5. Scope
6. Functional requirements (MoSCoW)
7. Non-functional requirements
8. User stories and acceptance criteria (excerpt)
9. Risks
10. Release roadmap
11. Assumptions

MovieOn — Phase 2: System Design

1. Architecture overview
2. Data model
3. REST API
4. Security design
5. Video delivery and playback authorisation
6. UI / UX design
7. Scalability path
8. Entitlements and billing design

MovieOn — Phase 4: Testing

1. Strategy
2. Requirements traceability
3. Current results (v1.1.1, 2026-10-20)
4. Test cases — representative detail
5. Performance measurement
6. Manual test checklist (before each release)
7. Known limitations of the test suite
8. Exit criteria for v1.1

MovieOn — Phases 5–6: Deployment & Maintenance

1. Environments
2. Configuration
3. Running
4. CI/CD pipeline
5. Release process

6. Database operations
7. Monitoring & alerting
8. Security operations
9. Content compliance & takedown
10. Maintenance plan
11. Roadmap and known gaps (v1.1 → next)
12. Payments and subscriptions — operating notes

MovieOn — Functions Overview

Version 1.1.1 · 2026-10-20

This document lists **every function the product provides**, grouped by who uses it, with the requirement ID (`FR-xx`, see `01-planning-and-requirements.md`) and what is — and is not — enforced. Screenshots are from the automated browser test (`e2e/smoke.mjs`) running against the real application; the test video is a generated colour-bar clip, not a film.

1. Who uses MovieOn

Actor	Can do
Visitor (not signed in)	Browse, search, open title pages, watch **free** titles, see plans
Viewer (signed in, Free plan)	Everything above + watchlist, ratings, saved progress / Continue watching, subscribe
Subscriber (Silver / Gold / Diamond)	Everything above + titles of the plan's tier and lower, more simultaneous screens, account page
Admin	Everything (admins are treated as Diamond) + back-office: titles, episodes, tiers, rights notes, VK embeds, grant / revoke plans
Operator	Deploys, configures (`BILLING_PROVIDER`, `COOKIE_SECURE`, …), backs up, monitors (see `04-deployment-and-operations.md`)

2. Plans and what they unlock

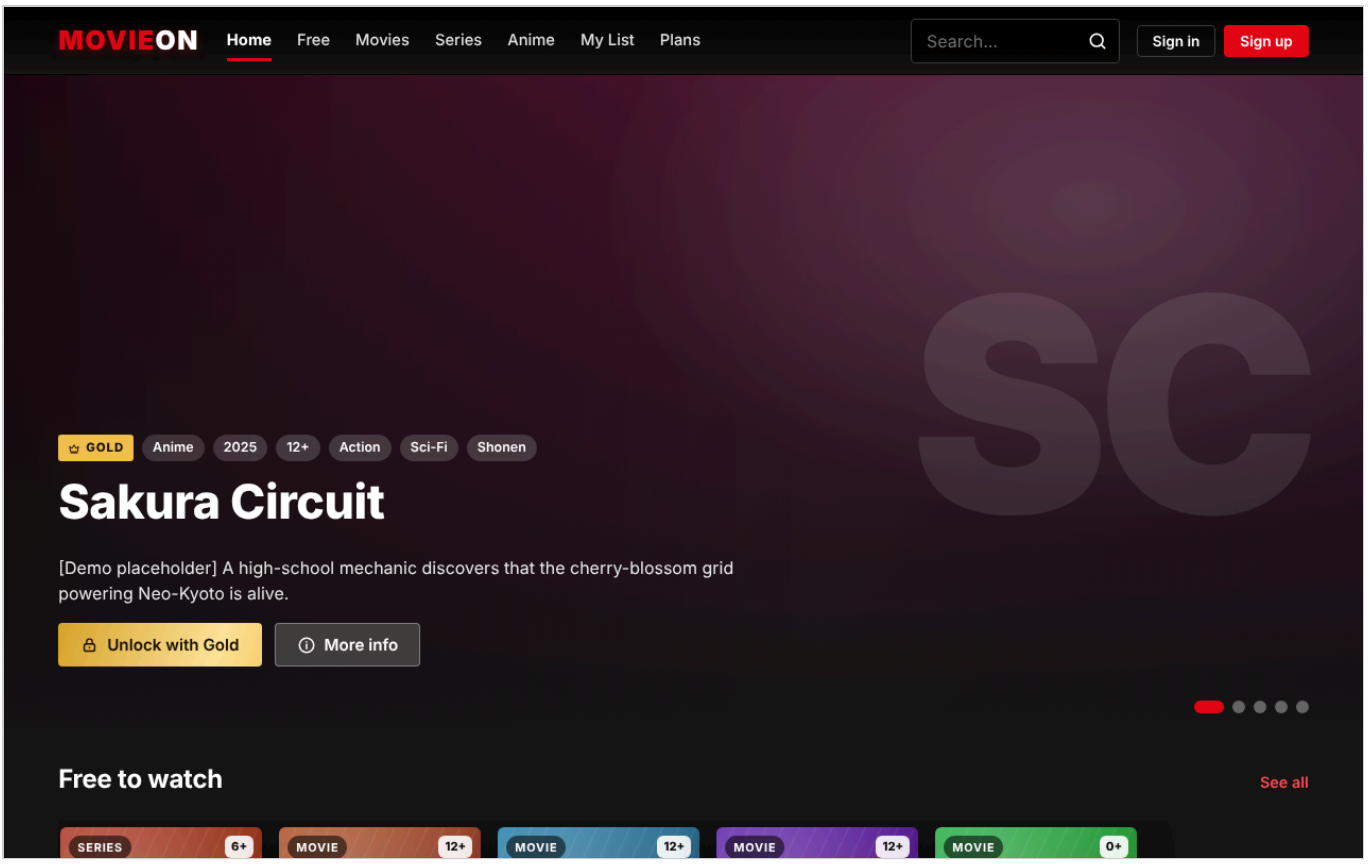
	Free	Silver	Gold	Diamond
Price (placeholder, USD / 30 days)	0	4.99	8.99	12.99
Titles of tier	Free	Free + Silver	+ Gold	+ Diamond
Simultaneous screens	1	1	2	4
Account needed to watch	free titles: no	yes	yes	yes

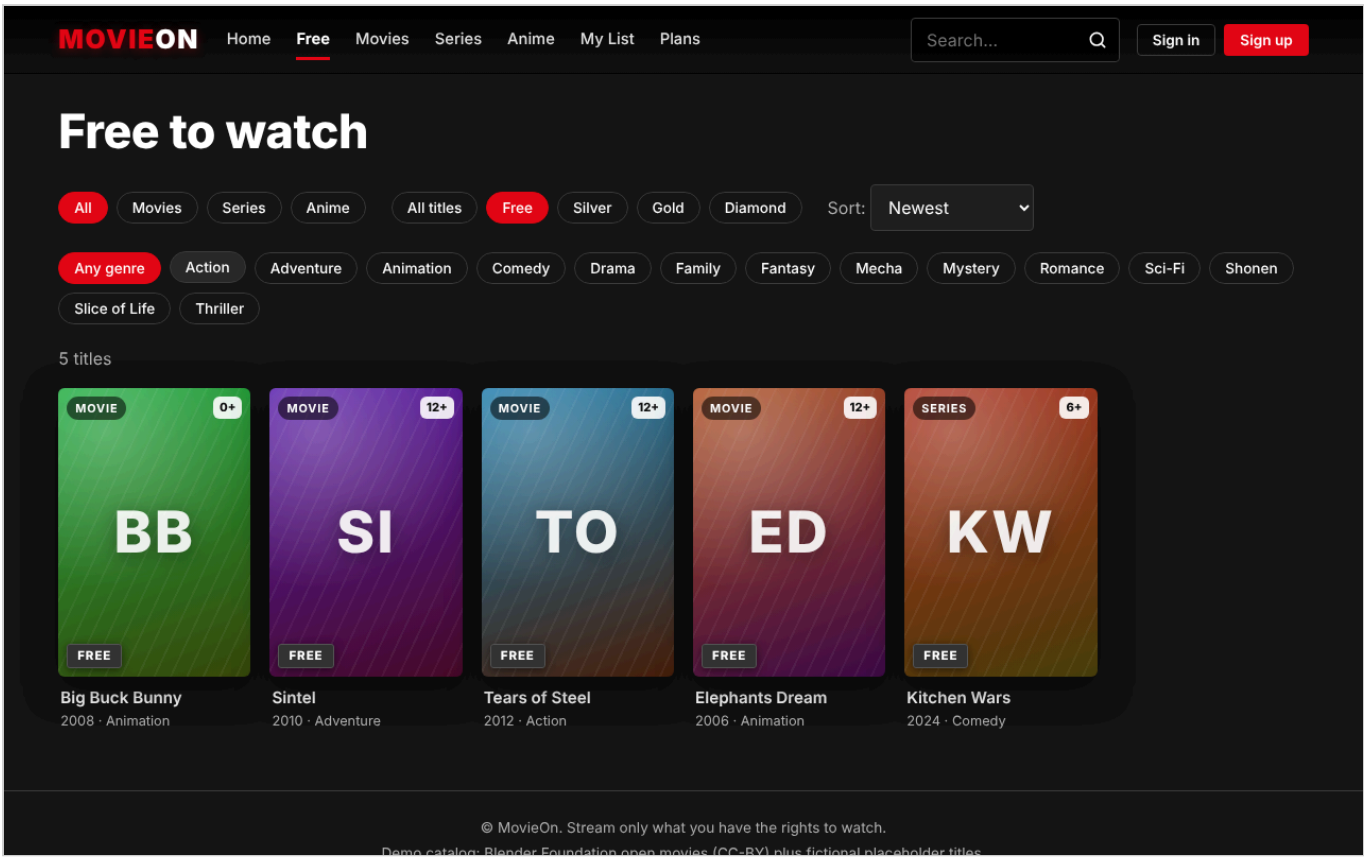
Enforced by the server. **Not** provided: picture-quality tiers (every title has one source), downloads, profiles. Prices and limits live in `server/src/plans.js`.

3. Viewer-facing functions

3.1 Discover

Function	Detail	FR
Home	Rotating hero (locked titles show "Unlock with [plan]"), rows: Continue watching (signed-in), Trending, New releases, **Free to watch**, Movies, Series, Anime	FR-03, FR-22
Browse	Filter by type, **access (All / Free / Silver / Gold / Diamond)** and genre; sort newest / most watched / top rated / A–Z; pagination	FR-04, FR-22
Search	Text search over title and synopsis; empty state; wildcard- and injection-safe	FR-05
Tier labels	Every poster shows Free / Silver / Gold / Diamond; locked posters are dimmed and carry a lock icon	FR-25

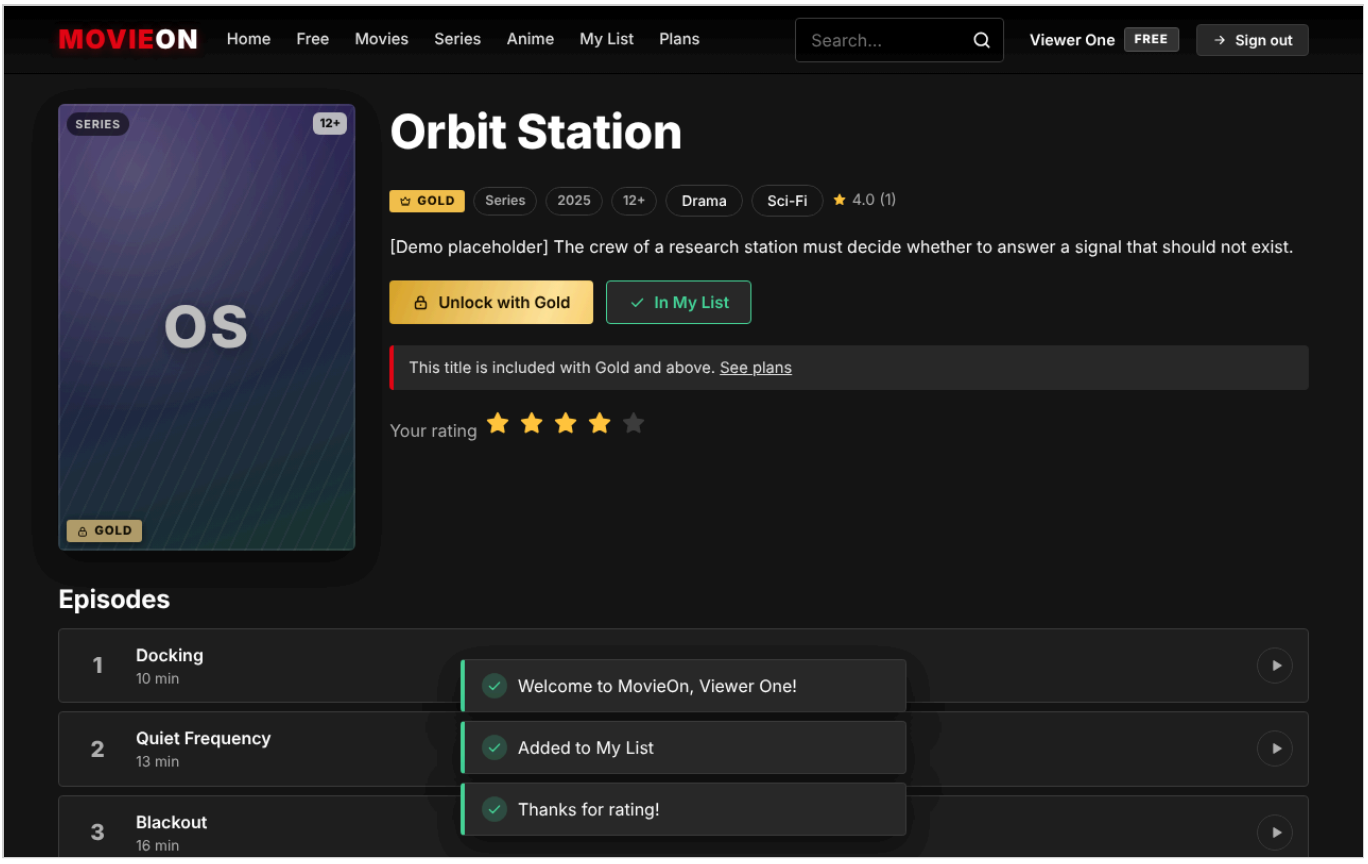




3.2 Title page

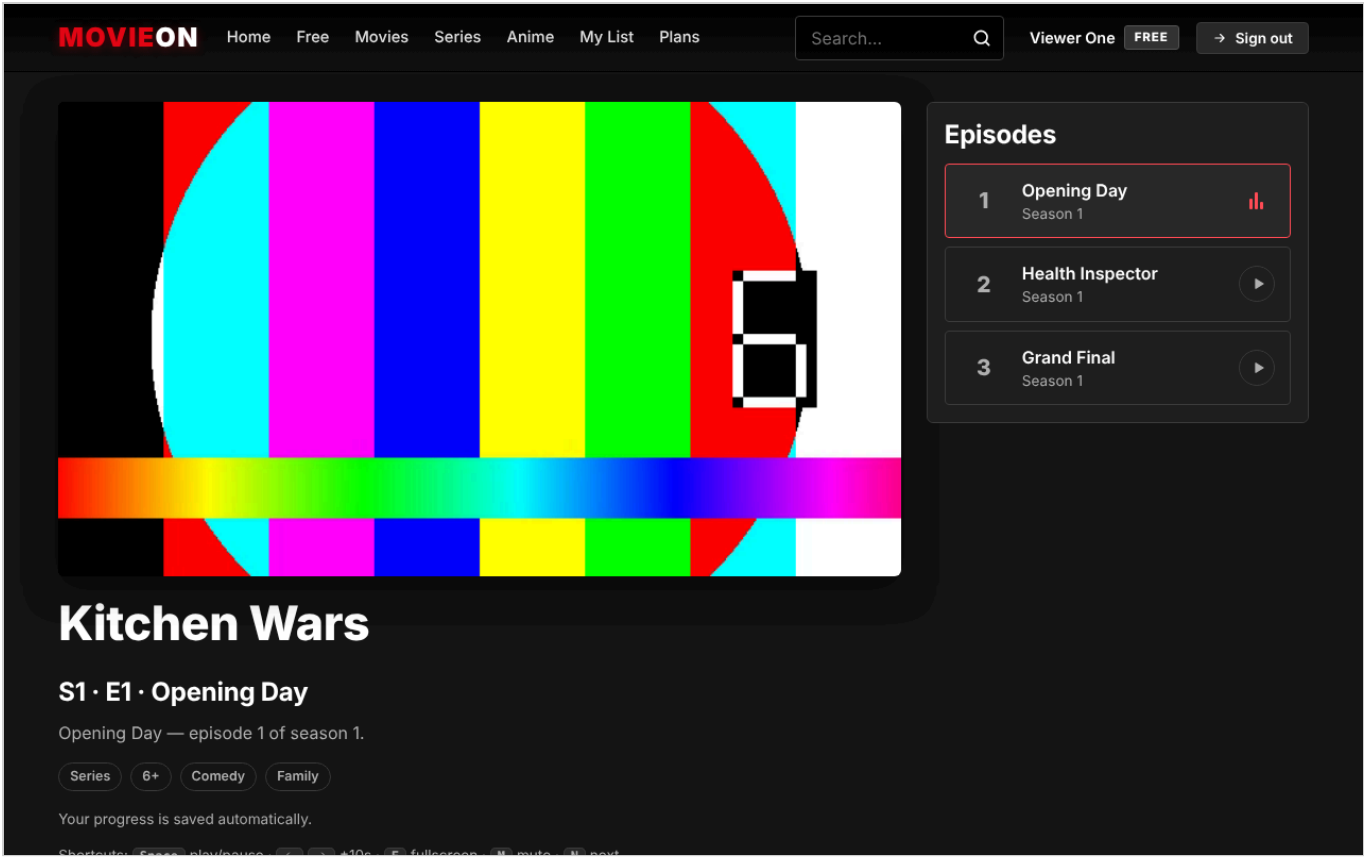
Function	Detail	FR
Details	Poster, tier, type, year, age rating, genres, synopsis, average rating	FR-06

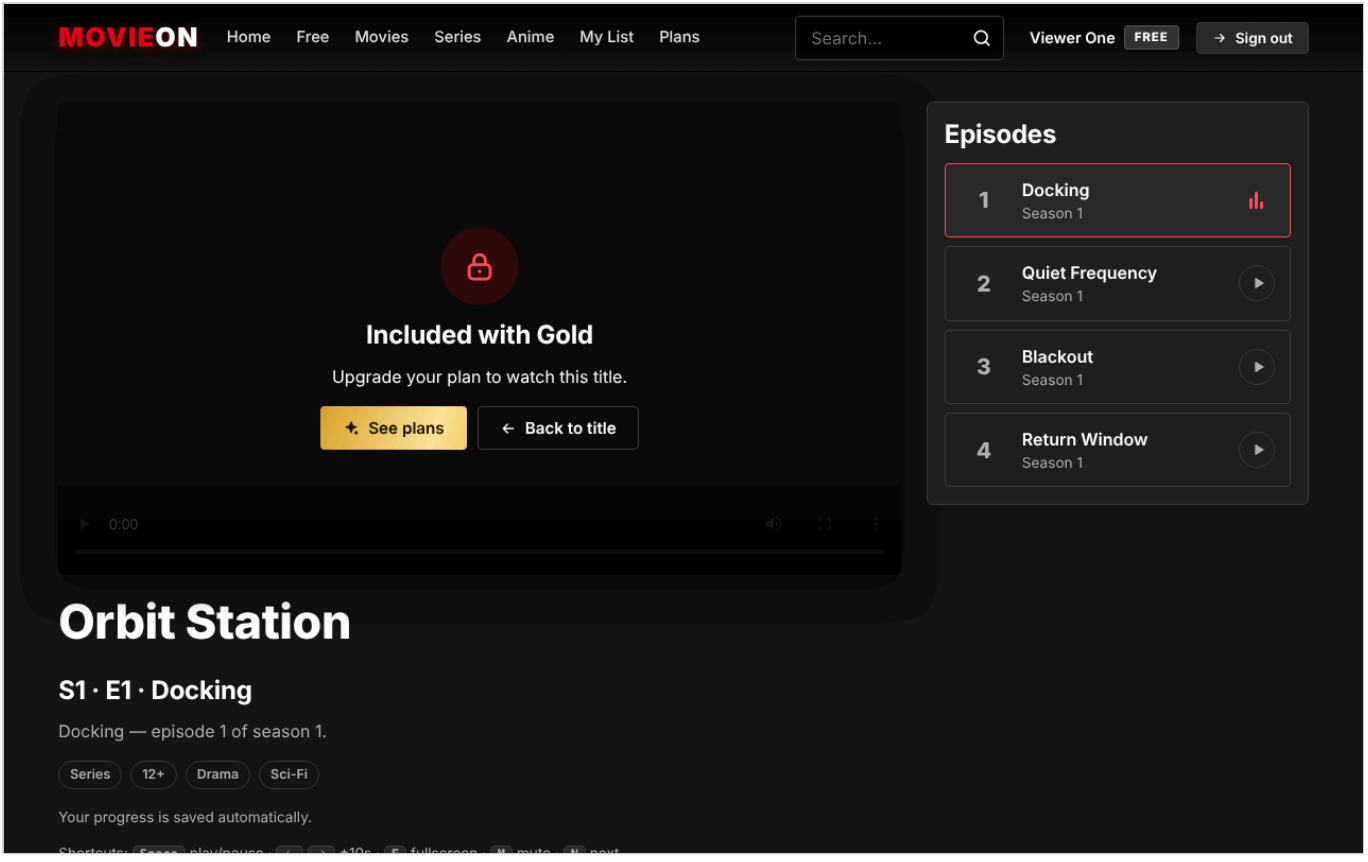
Function	Detail	FR
Episodes	Seasons and episodes with progress bars and "Watched" marks	FR-06
Primary action	Play / Resume from mm:ss / Continue S·E — or **Unlock with [plan]** when the plan does not cover the title	FR-08, FR-25
My List	Add / remove (sign-in required)	FR-09
Rating	1–5 stars, average and count	FR-10



3.3 Player

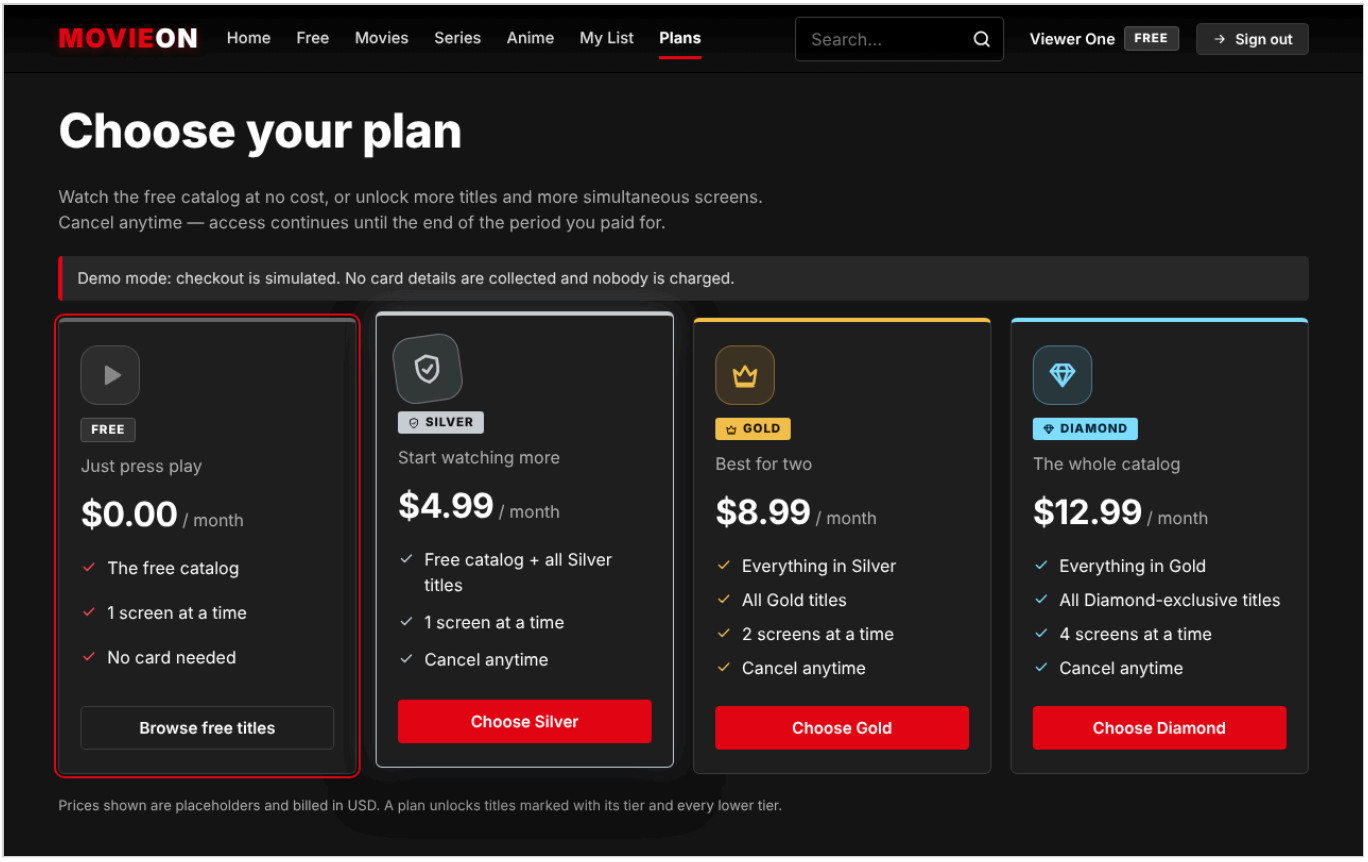
Function	Detail	FR
Playback	MP4 and HLS (`.m3u8`; hls.js or native Safari), native controls, fullscreen	FR-07
Authorisation	The video URL is requested from the server at play time; refusal reasons are explained: **sign in**, **upgrade**, **too many screens**	FR-19, FR-21, FR-25
Screen slots	Heartbeat every 30 s; slot freed on exit or 90 s after the last heartbeat; a tab switching episodes replaces its own slot	FR-21
Resume	Position saved every 10 s, on pause and on leaving; reopening continues from the saved point	FR-08
Next episode	Countdown overlay with Play now / Stay here	FR-11
Shortcuts	Space play/pause · ←/→ ±10 s · F fullscreen · M mute · N next	FR-15
Error handling	"Playback problem" panel with Retry (keeps the position)	FR-07
VK embeds	Official VK player in a sandboxed iframe; **no** resume, progress or shortcuts for these	FR-23

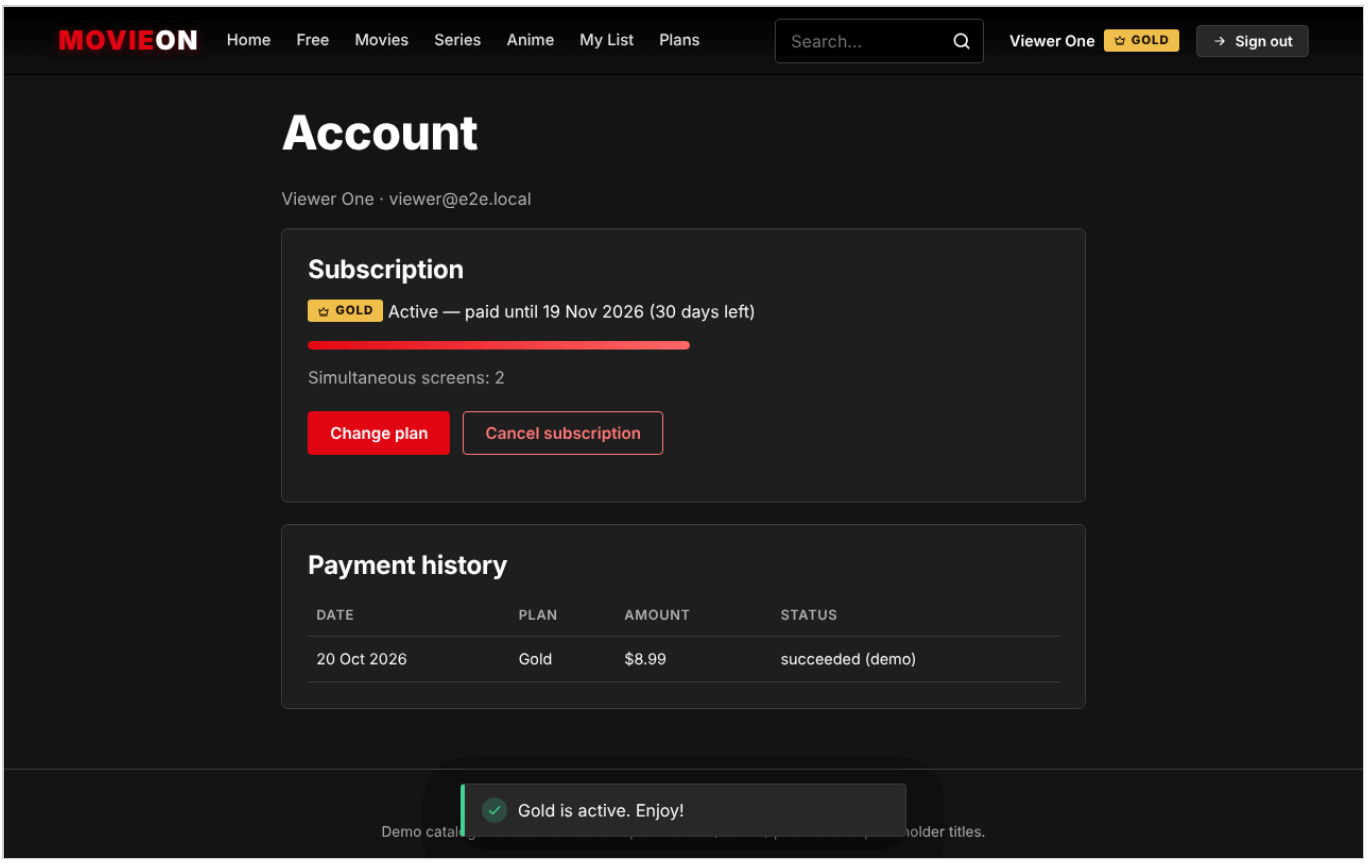

MOVIEON
Home
Free
Movies
Series
Anime
My List
Plans
Search...
Viewer One
FREE
Sign out
Episodes
1
Opening Day
Season 1
2
Health Inspector
Season 1
3
Grand Final
Season 1
Kitchen Wars
S1 · E1 · Opening Day
Opening Day — episode 1 of season 1.
Series
6+
Comedy
Family
Your progress is saved automatically.


MOVIEON
Home
Free
Movies
Series
Anime
My List
Plans
Search...
Viewer One
FREE
Sign out
Episodes
1
Docking
Season 1
2
Quiet Frequency
Season 1
3
Blackout
Season 1
4
Return Window
Season 1
Included with Gold
Upgrade your plan to watch this title.
See plans
Back to title
0:00
Orbit Station
S1 · E1 · Docking
Docking — episode 1 of season 1.
Series
12+
Drama
Sci-Fi
Your progress is saved automatically.

3.4 Account, plans and billing

Function	Detail	FR
Register / sign in / sign out	Email + display name + password (≥ 8 chars); httpOnly SameSite=Strict cookie, 7 days; generic "invalid credentials"	FR-01, FR-02
Plans page	Free + Silver / Gold / Diamond cards with price, screens, features; current plan highlighted	FR-18
Checkout	Confirm page; **demo provider**: records a payment, no card fields, no real charge; disabled (with an explanation) when `BILLING_PROVIDER=none`	FR-20
Change plan	Starts a new 30-day period at full price (no proration)	FR-20
Cancel / resume	Cancel stops renewal, access runs to the period end; resume while the period runs	FR-20
Account page	Plan, status and days left, simultaneous screens, payment history	FR-20
Expiry	After the period ends the user falls back to Free; a running stream is stopped at the next heartbeat	FR-19, FR-21





3.5 Design and accessibility

Red/black dark design with an own wordmark (no third-party logo), responsive from 360 px, keyboard accessible (skip link, visible focus, labelled controls, tier shown as text not only colour), no external fonts / CDNs / trackers (FR-14, FR-16, NFR-04, NFR-09).

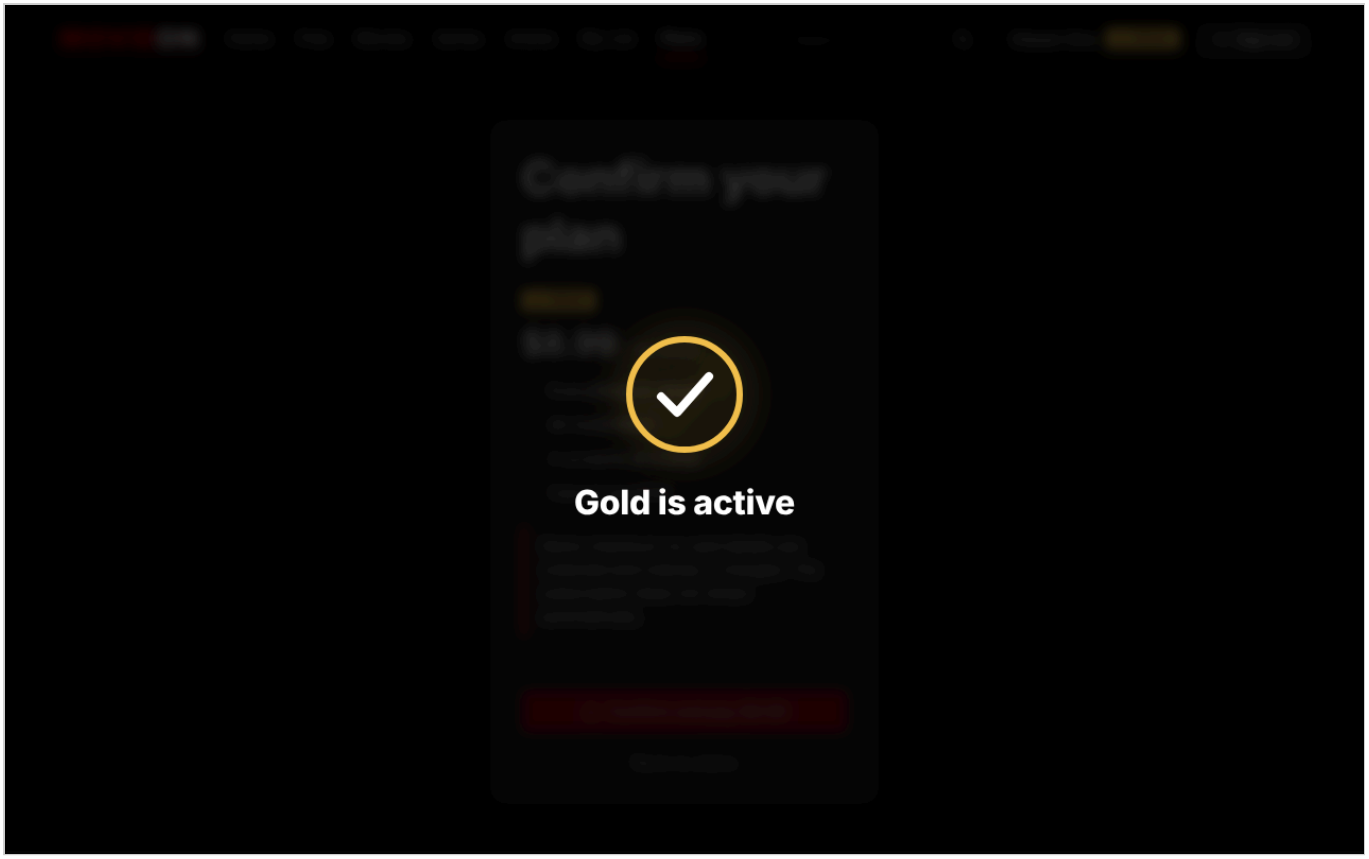
Custom icon set (v1.1.1, FR-26). All icons are original 24×24 SVGs drawn for this project (`web/js/icons.js`, 21 icons: play, lock, check, plus, close, chevrons, arrows, star, search, shield / crown / gem for Silver / Gold / Diamond, monitor, info, alert, sparkle, retry, film, user). They replace every emoji and text glyph (🔒 ▶ ✓ ★ × ← → ●), so the UI looks identical on every OS, inherits the text colour and can be animated. Icons are decorative (`aria-hidden`); the adjacent text is the accessible name.

Motion (v1.1.1, FR-27). Hand-written CSS and a few lines of JS, no animation library:

Where	Animation
App start	Intro splash: MOVIEON drawn letter by letter, red light line; **once per browser session**, click / key skips it
Every page	Thin red progress bar while a page loads; page fade-and-rise; header gains a solid backdrop on scroll; animated nav underline
Home	Hero background drift, floating giant initials, staggered text entrance, pulsing Play button, gold flowing "Unlock with..." button
Rows and grids	Cards rise in one after another; hover lifts the poster, zooms its pattern and fades in a Play (or Lock) button; row arrows appear on hover
Tier badges	Periodic sheen on Silver / Gold / Diamond; crown bobs, gem twinkles; the lock's shackle opens on hover
Title page	My List button swaps `+` for a drawn ✓ (icon); stars pop in a wave and preview on hover; progress bars grow; episode rows slide on hover
Plans	Staggered card entrance, tier-coloured glow and tilted emblem on hover, check marks drawn in sequence, glowing ring on the current plan
Checkout	**Celebration**: ring and tick drawn in the plan's colour, particle burst, then the account page

Where	Animation
Player	Loading spinner (ring + pulsing play glyph), shaking lock / pulsing monitor / alert icons on the gate screens, animated equaliser on the current episode, **circular 8-second countdown** to the next episode
Feedback	Toasts with an icon and a draining timer bar; ripple on buttons and chips; error shake on form messages; account period meter

Reduced motion. With the OS setting "reduce motion" the splash and hero rotation are off, every CSS animation and transition is disabled, and content simply appears; the loading spinner becomes a soft fade so that "busy" is still visible. This is covered by an automated test.





4. Admin functions

Function	Detail	FR
Titles	Create / edit / delete; type (movie, series, anime); synopsis, year, age rating, minutes, genres; published / draft; featured on home	FR-12
Tier	"Available on": Free, or Silver / Gold / Diamond and above	FR-17, FR-24
Sources	Direct MP4 / HLS URL, **or** the official VK embed link (VK page links are rejected, link is validated and rebuilt)	FR-23
Rights note	Mandatory for VK embeds: who owns / licenses the video and that embedding is permitted	FR-23
Episodes	Add (season, number, name, minutes, source) and delete; shows which are VK embeds	FR-12, FR-23
Publish / unpublish	Immediate takedown; drafts are invisible to the public	FR-12
Grant / revoke plan	By user email, plan and number of days (support, partners, pilots)	FR-24
Admin-only data	Admin list exposes the stored source URL and rights note, which no public endpoint does	FR-19

5. API functions (summary)

Area	Endpoints
Auth	`POST /auth/register`, `POST /auth/login`, `POST /auth/logout`, `GET /auth/me`
Catalog	`GET /home`, `GET /genres`, `GET /titles` (type, genre, q, tier, sort, page, limit), `GET /titles/:slug`
Playback	`POST /playback`, `POST /playback/:id/heartbeat`, `DELETE /playback/:id`
User data	`GET/PUT/DELETE /watchlist…`, `PUT /titles/:id/rating`, `GET /progress`, `GET /progress/:titleId`, `PUT /progress`
Billing	`GET /plans`, `GET /billing/subscription`, `POST /billing/checkout`, `POST /billing/cancel`, `POST /billing/resume`
Admin	`GET/POST /admin/titles`, `PUT/DELETE /admin/titles/:id`, `GET/POST /admin/titles/:id/episodes`, `DELETE /admin/episodes/:id`, `POST /admin/subscriptions`, `POST /admin/subscriptions/revoke`
System	`GET /health`

Full contracts, validation rules and error codes: `02-design.md` §3.

6. System functions (no UI)

Function	Detail
Schema migrations	Ordered, transactional, tracked with `PRAGMA user_version`; v1.0 databases upgrade in place
Security headers	helmet CSP (`script-src 'self'`, `frame-src` limited to the VK embed hosts, `frame-ancestors 'none'`), HSTS only over HTTPS
Rate limiting	20 auth attempts / 15 min / IP; 300 requests / min / IP globally
CSRF defence	SameSite=Strict cookie, JSON-only bodies, `Origin` check on mutating requests
Structured logs	One JSON line per request (no query strings or bodies)
Demo seed	13 titles: 4 Blender open movies (CC-BY) + fictional placeholders, spread over all tiers; `SEED_DEMO=false` for an empty start
CLI	`npm run admin -- <email> <password>` creates or promotes an admin

7. Known boundaries (read before launching)

1. **Payments are a demo.** No real money is processed; there is no renewal, proration, invoice or tax logic.
2. **A video URL that was issued can be copied.** Tier checks stop casual bypass; real protection needs signed URLs / DRM (roadmap).
3. **VK:** only the official embed, only for videos you have the right to show; no link extraction. Availability and tracking are VK's.
4. **Prices, plan names and limits are placeholders** to be set by the operator.
5. English-only UI; no subtitles, profiles, parental controls or password reset yet (see roadmap).

MovieOn — Phase 1: Planning & Requirements

Version 1.1.1 · 2026-10-20

> **Changes in 1.1.1:** original SVG icon set replacing emoji/glyph icons (FR-26) and a full motion layer — splash, page/entrance/hover animations, celebration, player loader, countdown ring — that honours `prefers-reduced-motion` (FR-27). No back-end or data changes.
> **Changes in 1.1:** content tiers and subscription plans (Silver / Gold / Diamond), server-side entitlement checks and simultaneous-screen limits, a free catalog, official VK embed support (rights-note gated), red/black visual redesign (light theme removed), numbered database migrations. Payments are a **demo provider only** — see §2 and §9.

1. Vision

MovieOn is a web streaming platform where people watch **movies, series and anime** in one catalog, with a clean player, a personal watchlist and automatic "continue watching".

Product statement: *For viewers who juggle several apps to follow films, shows and anime, MovieOn is a single, fast, ad-light catalog with resume-anywhere playback — a free tier to start and Silver / Gold / Diamond plans for more titles and more screens. Unlike piracy aggregators, MovieOn only streams content the operator is licensed to distribute.*

Business model (v1.1): freemium. A free catalog (watchable without an account) drives acquisition; three paid plans unlock higher content tiers and more simultaneous screens. Plan prices in this repository are **placeholders in USD** to be set by the operator.

2. Content and legal model (critical constraint)

A streaming site is only viable if the operator holds distribution rights. This shapes the product:

Topic	Decision
Content source	Operator-owned, licensed (distributor/aggregator agreements) or openly licensed (e.g. Blender Foundation open movies, CC-BY, public domain).
Pirated content	Out of scope. The platform does not crawl, embed or proxy third-party pirate streams.
Third-party hosted video (VK)	**Decision (v1.1):** the platform does **not** scrape or extract direct stream links from VK or any other site (that circumvents the host's player and terms, and most "free movies" uploaded by users are unlicensed). Instead an admin may attach VK's **official embed** (`video_ext.php`) for a video the operator owns or is licensed/permitted to show; a **rights note** (who owns it / what permits embedding) is mandatory and stored with the title. VK page links are rejected. Rights are the operator's responsibility; the embed can disappear if the uploader removes the video or disables embedding.
Payments	v1.1 ships a **demo billing provider** that records a successful payment without collecting any card data. A real provider (hosted checkout + signed webhooks) is required before charging anyone — see roadmap. The platform never stores card numbers.
Prices	Placeholder USD prices (Silver 4.99, Gold 8.99, Diamond 12.99 per 30 days) defined in `server/src/plans.js`. Tax/VAT, invoicing, refunds and local-currency pricing are not implemented.
Demo data	The seed catalog uses openly licensed Blender films plus clearly fictional placeholder titles that reuse those sample videos. Real rights must be added before launch.
Takedown	Admin can unpublish/delete any title immediately; a DMCA/notice-and-takedown process is defined in the operations doc.
Geo-rights	Not implemented; listed in the roadmap (needed for real licensing deals).
Age ratings	Every title carries an age rating label (`0+`, `6+`, `12+`, `16+`, `18+`). Parental controls are in the roadmap.

Topic	Decision
Privacy	Minimal personal data: email, username, password hash, watch history, subscription and payment records (plan, amount, date). GDPR-style export/delete is a roadmap item. Opening a VK embed loads VK's player and therefore VK's own cookies/tracking for that page — disclose in the privacy policy.

3. SDLC model

Iterative Agile (Scrum-lite), 2-week sprints, with a thin waterfall-style documentation gate at the start (this folder). Rationale: requirements around content and licensing change, UX needs fast feedback, but the data model and security posture must be settled early.

Phase	Output	Where
0. Functions overview	Every function by actor, plans, boundaries	`00-functions-overview.md`
1. Planning & requirements	Vision, scope, requirements, risks	this document
2. Design	Architecture, data model, API, security, UI	`02-design.md`
3. Implementation	Server, web client, seed data	`server/`, `web/`
4. Testing	Strategy, traceability, automated suite	`03-testing.md`, `server/test/`
5. Deployment	Docker, CI/CD, config, runbook	`04-deployment-and-operations.md`
6. Maintenance	Monitoring, backups, release cadence, roadmap	`04-deployment-and-operations.md`

Definition of Done per story: code merged via PR, automated tests pass in CI, linted, documented in API table if an endpoint changed, accessibility check on touched screens, no new high-severity `npm audit` findings.

4. Stakeholders and personas

Persona	Goal	Pain today
Aigul, 24, student — watches anime and series on her phone	Find something quickly, pick up where she stopped	Many apps, no shared history
Rustam, 38, parent	Browse by genre and age rating	Unclear ratings, cluttered UI
Dilnoza, content manager (admin)	Add titles/episodes, remove titles on request	Needs a simple back-office, not SQL
Operator/owner	Low run cost, legal safety, growth metrics	Hosting and licensing cost

5. Scope

In scope (v1.0 MVP): accounts, catalog (movies/series/anime), genres, search, details with seasons/episodes, MP4 and HLS playback, resume, next-episode, watchlist, 1–5 star ratings, trending by views, admin back-office, responsive UI, REST API, tests, Docker, CI.

Added in v1.1: content tiers (Free / Silver / Gold / Diamond), subscription plans with account page (cancel / resume / payment history), demo checkout, server-side playback authorisation, simultaneous-screen limits, free catalog row and filter, official VK embed with rights note, admin tier / rights / grant-revoke tools, red/black Netflix-style visual design, numbered DB migrations.

Out of scope (v1.1): a real payment provider, automatic renewal, proration, invoices/tax, DRM, live TV, downloads/offline, comments, recommendations ML, multi-profile, native apps, subtitles upload pipeline, transcoding service, geo-restriction, **picture-quality (resolution) tiers** — plans differ by catalog access and screens only, because each title is a single source.

6. Functional requirements (MoSCoW)

ID	Requirement	Priority
FR-01	Visitor can register with email, username, password (min 8 chars).	Must
FR-02	User can log in, log out, and stay signed in via secure cookie session (7 days).	Must
FR-03	Home page shows a featured hero and rows: Trending, New releases, Movies, Series, Anime, and Continue Watching (signed-in).	Must
FR-04	Catalog can be filtered by type, genre and sorted (newest, name, rating, popular).	Must
FR-05	Search by title/synopsis text with results page.	Must
FR-06	Title page shows poster, synopsis, year, age rating, genres, average rating, episodes grouped by season.	Must
FR-07	Player plays MP4 and HLS (`.m3u8`) sources in the browser, with native controls and fullscreen.	Must
FR-08	Playback position is saved periodically and restored on return; Continue Watching lists unfinished items.	Must
FR-09	User can add/remove titles to a personal watchlist.	Must
FR-10	User can rate a title 1–5 stars; average and count are displayed.	Should
FR-11	When an episode ends, the player offers/auto-plays the next episode.	Should
FR-12	Admin can create, edit, delete titles and add/delete episodes; set featured/draft.	Must
FR-13	"Trending" is computed from view counts (a view = an authorised playback start, counted server-side).	Should
FR-14	UI is responsive from 360 px to 4K and keyboard accessible.	Must
FR-15	Keyboard shortcuts in player: Space, F, M, ←/→ (10 s), N (next episode).	Could
FR-16	*(changed in 1.1)* Single dark red/black visual design in the style of mainstream streaming apps (own wordmark, no third-party logos). The light theme and toggle of v1.0 were removed.	Should
FR-17	Every title has a minimum tier: `free`, `silver`, `gold` or `diamond`. Free titles can be watched without an account.	Must
FR-18	Three paid plans — **Silver** (1 screen), **Gold** (2 screens), **Diamond** (4 screens). A plan unlocks its own tier and all lower tiers. Free viewers get 1 screen.	Must
FR-19	Entitlements are enforced **server-side**: video URLs are absent from all public payloads and are returned only by `POST /api/playback` after the tier and screen checks.	Must
FR-20	Signed-in user can subscribe (checkout), change plan, cancel (access continues to period end), resume a canceled plan, and see status and payment history on an account page. In v1.1 checkout is backed by a demo provider.	Must
FR-21	Simultaneous screens are limited per plan using playback sessions with a 30 s heartbeat and 90 s expiry; a tab starting a new playback replaces its own session.	Must
FR-22	Visitors can browse the free catalog: "Free to watch" row on home, `Free` filter (and Silver/Gold/Diamond filters) in Browse.	Should
FR-23	Admin can attach a VK official embed link to a movie/episode; the link is validated and normalised, VK page links are rejected, and a rights note is required.	Should
FR-24	Admin can set a title's tier and rights note and can grant or revoke a subscription for a user by email.	Must
FR-25	Locked titles are labelled (tier badge, lock) and offer an "Unlock with [plan]" action; the player explains login / upgrade / screen-limit situations.	Should
FR-26	*(1.1.1)* The UI uses one original SVG icon set instead of emoji or text glyphs; icons inherit the text colour, are decorative (`aria-hidden`) and never the only carrier of meaning.	Should
FR-27	*(1.1.1)* Custom motion: intro splash (once per session), page / list entrance, hover and press feedback, drawn check marks, plan-activation celebration, player loading spinner and next-episode countdown ring. With `prefers-reduced-motion: reduce` nothing animates and the splash is skipped. Only `transform`, `opacity`, `stroke-dashoffset` and `box-shadow` are animated; no animation library and no inline `style` attributes (CSP).	Should

7. Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	Catalog/list API p95 < 200 ms at 100 req/s on 1 vCPU / 512 MB; first contentful paint < 2 s on 4G for the home page.
NFR-02	Security	Passwords hashed with bcrypt (cost ≥ 10); httpOnly, SameSite=Strict session cookie; rate-limited auth; strict CSP (framing limited to the VK embed hosts); all SQL parameterised; no user data injected through `innerHTML`; entitlement decisions made only on the server; no card data handled.
NFR-03	Availability	99.5 % monthly for the web/API tier (excluding third-party video hosts).
NFR-04	Accessibility	Target WCAG 2.1 AA: focus visible, labels, contrast ≥ 4.5:1, reduced-motion respected.
NFR-05	Compatibility	Latest 2 versions of Chrome, Edge, Firefox, Safari; iOS Safari plays HLS natively.
NFR-06	Maintainability	≥ 80 % of API routes covered by automated tests; ES modules; no build step for the client.
NFR-07	Portability	Runs from a single container; SQLite for MVP with a documented path to PostgreSQL.
NFR-08	Observability	Structured request logs, `/api/health` endpoint.
NFR-09	Privacy	No third-party trackers; no external fonts or CDNs on the critical path (a VK embed is the one third-party player, loaded only on titles that use it).
NFR-10	Localisation	Target: translations (ru, uz). v1.1 still ships English only with UI strings inline in the page modules; extracting them into a dictionary is the first step of that work.
NFR-11	Data evolution	Schema changes are applied by ordered, transactional migrations tracked with `PRAGMA user_version`; an existing v1.0 database upgrades in place without data loss.

8. User stories and acceptance criteria (excerpt)

US-1 (FR-08) As a viewer I want playback to resume so I don't search for my place.

- Given I watched 12:40 of an episode and left, when I reopen it, then the player starts at ≈12:40.
- Given I finished (≥ 95 %), then it disappears from Continue Watching and progress resets for the next episode.

US-2 (FR-04/05) As a viewer I want to filter anime by genre.

- Given type = anime and genre = Action, then only anime tagged Action are listed, paginated.
- Given a query with no match, then an explicit empty state is shown.

US-3 (FR-12) As an admin I want to add a series with episodes.

- Given I am admin, when I submit the title form, then it appears in the catalog (if published) and I can add episodes with season/number.
- Given I am a normal user, then admin endpoints return 403 and the Admin link is hidden.

US-4 (FR-02/NFR-02) As a user I want a secure login.

- Given 21 failed requests from one IP in 15 minutes, then further attempts get 429.
- Given an invalid password, then the message does not reveal whether the email exists.

US-5 (FR-17/18/19/20) As a viewer I want to unlock more titles with a plan.

- Given I am on the Free plan and open a Gold title, then I see a Gold badge and "Unlock with Gold", and the player refuses to start (403 `upgrade_required`) without ever receiving the video URL.
- Given I confirm checkout for Gold, then I can play Gold and Silver titles immediately and my account page shows the plan, end date and the payment.
- Given I cancel, then I keep access until the paid period ends; after that Gold titles are locked again.

US-6 (FR-21) As an operator I want plans to limit simultaneous screens.

- Given a Silver subscriber is watching on one device, when a second device starts playback, then it is refused with `screen_limit` (1 screen); a Gold subscriber can use two screens, Diamond four.
- Given a device stops sending heartbeats for 90 s (closed laptop), then its slot is freed.

US-7 (FR-23) As an admin I want to show a video I have rights to from VK.

- Given I paste an ordinary `vk.com/video…` page link, then it is rejected with instructions to use the official embed link.
- Given I paste the embed link but leave the rights note empty, then saving fails; with a rights note it is stored normalised and plays in VK's own player (progress/Continue watching are not tracked for embeds).

9. Risks

#	Risk	Likelihood	Impact	Mitigation
R1	No distribution rights / takedown claims	High	Critical	Licensed or open content only; takedown process; admin delete
R2	Video bandwidth cost	Med	High	HLS via CDN, adaptive bitrate, cache headers; third-party hosting in MVP
R3	Third-party sample URLs go offline	Med	Low	Demo only; documented; player shows an error state
R4	Account abuse / credential stuffing	Med	Med	Rate limits, bcrypt, generic errors; 2FA in roadmap
R5	SQLite write contention at scale	Low (MVP)	Med	WAL mode; migration plan to PostgreSQL
R6	Scope creep (payments, DRM)	High	Med	Explicit out-of-scope list; roadmap gating
R7	Minors viewing mature content	Med	High	Age labels now; parental PIN/profiles in v1.3
R8	Charging real money with the demo provider or mishandling card data	Low	Critical	Demo provider collects no card data and is disabled by default in production (`BILLING_PROVIDER=none`); real provider must use a hosted checkout + signed webhooks so card data never touches this server
R9	A paying user copies the video URL once issued and shares it	Med	Med	URL is only issued after checks, but cannot be un-issued. Real protection = short-lived signed CDN URLs (v1.4) and DRM (v2.0); accepted for v1.1 and documented
R10	Embedded VK video removed, restricted or claimed	Med	Med	Mandatory rights note; player shows VK's own error; periodic manual review; prefer self-hosted licensed files
R11	Placeholder prices/plan limits mistaken for a final offer	Med	Low	Marked as placeholders in docs and checkout; single source of truth `plans.js`
R12	Subscription state drifts from a real provider (manual grants, expiry)	Med	Med	Provider webhook is the only writer in production (roadmap); admin grants are labelled `provider='admin'`

10. Release roadmap

Release	Content
v1.0 MVP	Catalog, player, resume, watchlist, ratings, admin, Docker, CI
v1.1	Tiers and plans (Silver/Gold/Diamond), entitlement enforcement, screen limits, demo checkout + account, free catalog, VK official embed, red/black redesign, numbered migrations

Release	Content
v1.1.1 (this repo)	Custom SVG icon set, full motion layer, reduced-motion support
v1.2	**Real payments:** hosted checkout + signed webhooks (candidates to evaluate: an international provider such as Stripe; regional providers such as Payme/Click for Uzbekistan — none is integrated or verified here), auto-renewal, proration, invoices/tax, dunning; subtitles (WebVTT), GDPR export/delete, email verification, password reset, ru/uz translations
v1.3	Multi-profile + parental PIN, PostgreSQL, Redis cache, Elasticsearch/pg_trgm search
v1.4	Upload + transcoding pipeline (FFmpeg → HLS ladder → object storage/CDN) with signed expiring URLs, geo-rights, optional resolution tiers
v2.0	DRM (Widevine/FairPlay), recommendations, native apps

11. Assumptions

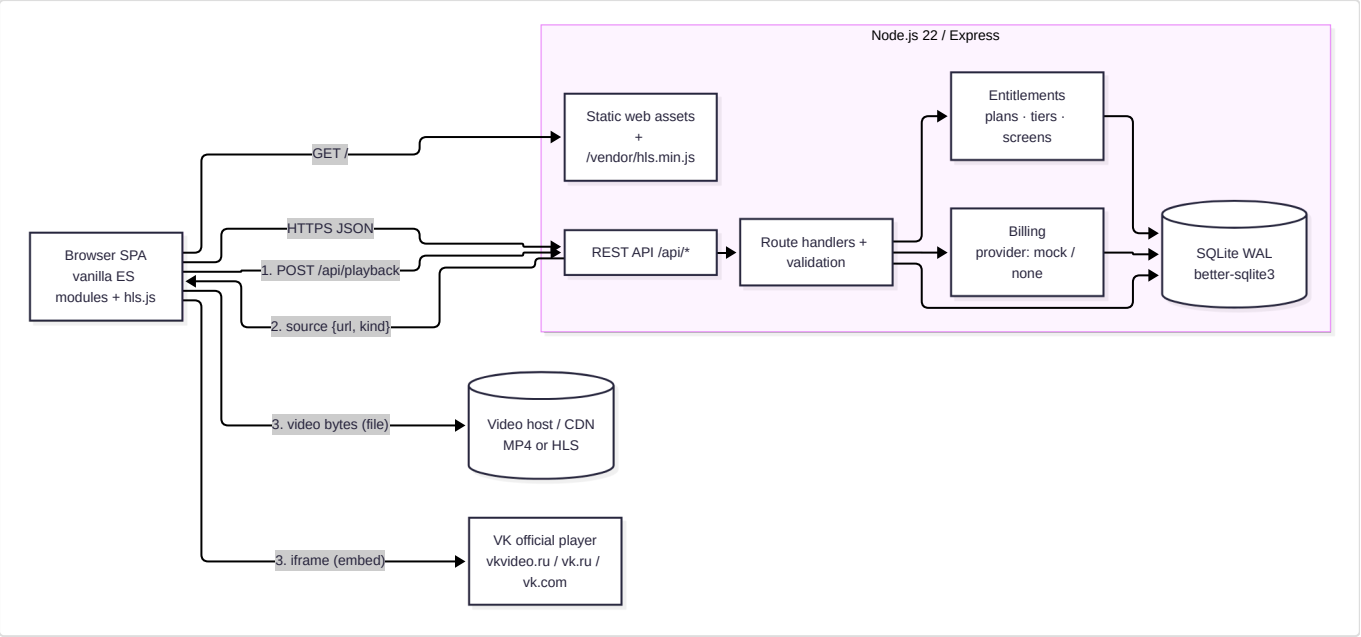
- Node.js 22 LTS runtime; operator provides HTTPS termination (reverse proxy or platform).
- Poster/backdrop art in the MVP is generated from per-title colour pairs; real artwork URLs can replace it via the admin form later.

MovieOn — Phase 2: System Design

Version 1.1.1 · 2026-10-20

> **Changes in 1.1.1:** client only — `icons.js` (SVG icon set), SVG-aware DOM helper, motion layer in `styles.css`, splash / ripple / scroll helpers in `main.js`, route progress bar in `router.js`. See §6 "Icons" and "Motion".
> **Changes in 1.1:** entitlements (tiers/plans), playback authorisation endpoint, playback sessions (screen limits), billing with a provider abstraction (demo provider only), VK embed sources, red/black UI, schema migrations V1→V2.

1. Architecture overview



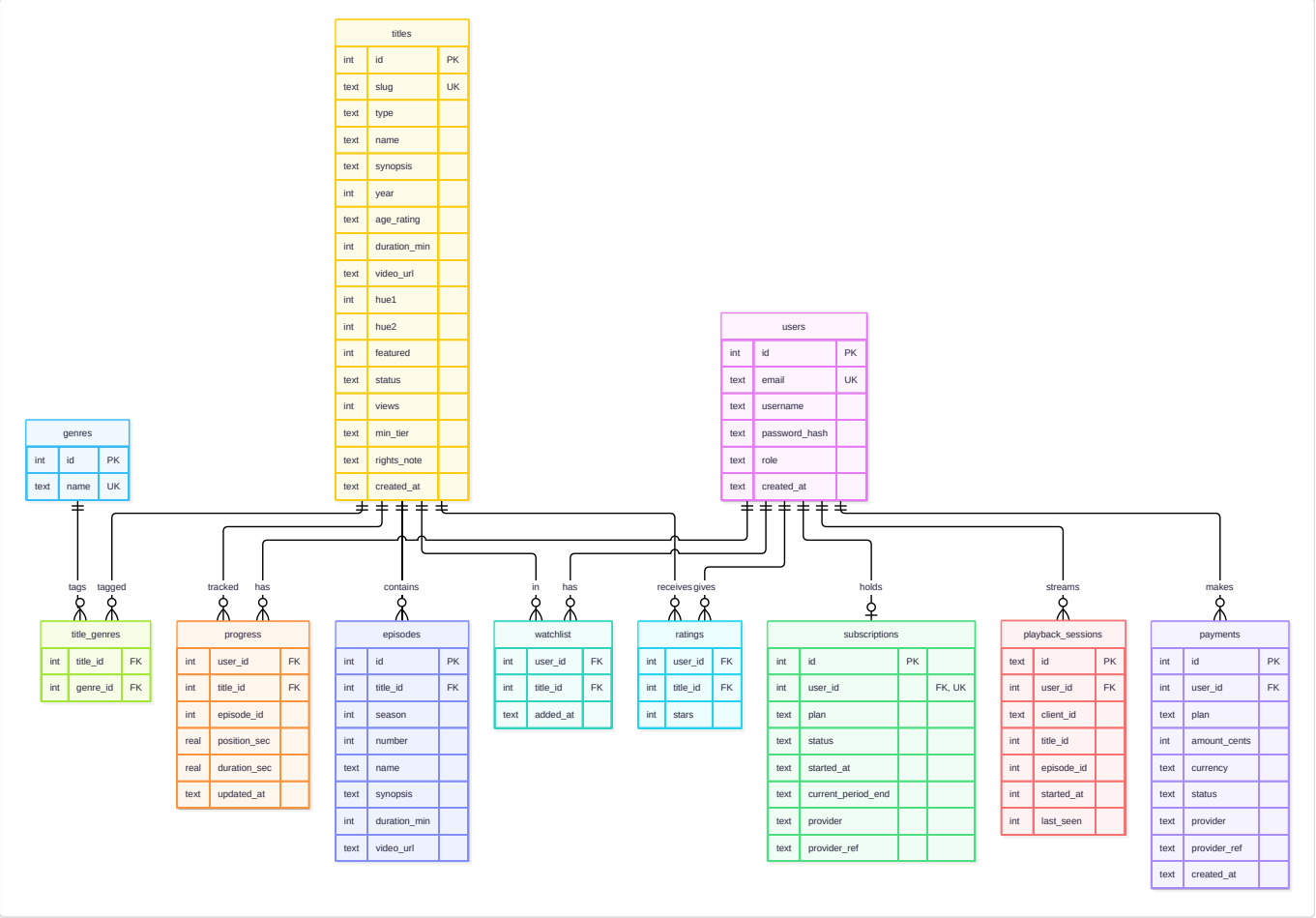
- **Monolith** (one deployable) for the MVP: lowest operational cost, simple to reason about.
- **Video bytes never pass through the API.** The client fetches MP4/HLS directly from a video host/CDN (or shows VK's own player in an iframe). The API stores the URL, decides **who may receive it** (tier + screens) and tracks playback progress.
- **No client build step.** Native ES modules keep the repo small and CI fast; hls.js is vendored from npm and served from `/vendor/` so CSP can stay `script-src 'self'`.

Technology choices and rationale (ADR summary)

ADR	Decision	Alternatives	Why
ADR-1	Express 5 on Node 22	Fastify, NestJS	Ubiquitous, tiny surface, fits MVP
ADR-2	SQLite (better-sqlite3, WAL)	PostgreSQL	Zero-ops for MVP; schema is portable SQL; migration path documented
ADR-3	Cookie session (JWT in httpOnly cookie)	Bearer token in localStorage	Not readable by injected scripts; SameSite=Strict mitigates CSRF
ADR-4	Vanilla JS SPA, hash routing	React/Vite	No toolchain, works on any static host; migrate if UI complexity grows
ADR-5	hls.js + native `<video>`	Video.js, Shaka	Smallest dependency covering HLS; Safari uses native HLS

ADR	Decision	Alternatives	Why
ADR-6	Generated gradient posters	Stored images	No copyrighted artwork in repo; URL override planned
ADR-7	DOM built with a helper, never `innerHTML` with data	Template strings	Eliminates a whole XSS class
ADR-8	Entitlements enforced on the server; video URLs withheld from public payloads and issued by `POST /api/playback`	Client-side hiding of locked titles	A client-side check is cosmetic; the URL must not exist in the response of a locked title
ADR-9	Billing behind a provider switch (`BILLING_PROVIDER` = `mock` | `none`); subscriptions written only by the billing layer or by an admin	Hard-wiring a payment SDK now	Lets the whole subscription flow be built and tested without credentials or card data; a real provider only needs to replace the checkout and write the same rows from a webhook
ADR-10	VK supported **only** through its official embed player, with a mandatory rights note	Extracting direct stream URLs from VK	Extraction bypasses VK's player/terms and encourages unlicensed content; the embed is a supported, documented mechanism
ADR-11	Numbered migrations (`PRAGMA user_version`)	`CREATE TABLE IF NOT EXISTS` only	Required once the schema changes (V2); keeps old databases upgradeable
ADR-12	Dark-only red/black design	Dual theme	One carefully checked palette instead of two; matches the product's cinematic positioning

2. Data model



Constraints: `titles.type IN ('movie','series','anime')`; `titles.status IN ('published','draft')`; `titles.min_tier IN ('free','silver','gold','diamond')` default `free`; `episodes UNIQUE(title_id, season, number)`; `ratings.stars BETWEEN 1 AND 5`; `progress` PK `(user_id, title_id, episode_id)` with `episode_id = 0` for movies; `subscriptions.user_id` UNIQUE (one row per user), `plan IN ('silver','gold','diamond')`,

`status IN ('active','canceled')`; foreign keys `ON DELETE CASCADE`. Indexes on `titles(type,status)`, `titles(views)`, `titles(created_at)`, `episodes(title_id)`, `payments(user_id, id)` and `playback_sessions(user_id, last_seen)`.

Movies carry `video_url` on the title. Series and anime carry URLs on episodes. A URL is either a direct file (MP4 / HLS) or a normalised VK embed link (`kind = embed`); `rights_note` is mandatory for embeds.

Migrations. `server/src/db.js` holds an ordered list: **V1** = the original schema (idempotent), **V2** = tiers, rights note, subscriptions, payments, playback sessions. On start, `migrate()` reads `PRAGMA user_version`, applies each missing migration in its own transaction and bumps the version. A v1.0 database (version 0) is upgraded in place; existing titles become `free`. Covered by an automated test.

3. REST API

Base path `/api`. JSON in/out. Auth is the `mo_token` cookie. Errors: `{ "error": "message", "code"?: "…" }` with proper status codes (400 validation, 401 unauthenticated, 403 forbidden, 404, 409 conflict, 429 rate limit, 503 feature disabled). Machine-readable `code` values used by the client: `login_required` (401), `upgrade_required` (403, with `requiredTier`), `screen_limit` (403, with `screens`), `session_expired` (404), `billing_disabled` (503), `already_subscribed` (409).

Rows in **bold** are new in v1.1.

Method & path	Auth	Description
GET `/health`	–	Liveness + DB check
POST `/auth/register`	–	`{email, username, password}` → 201 `{user, access}` + cookie
POST `/auth/login`	–	`{email, password}` → `{user, access}` + cookie
POST `/auth/logout`	–	Clears cookie
GET `/auth/me`	optional	`{user, access}`; `access = {level, plan, screens, subscription}`
GET `/genres`	–	Genre list
GET `/home`	optional	Hero + rows (trending, new, **free to watch**, movies, series, anime, continue)
GET `/titles`	–	Query: `type, genre, q, tier(free|silver|gold|diamond), sort(newest|name|rating|popular), page, limit` → `{items,total,page,pages}`; items carry `minTier`
GET `/titles/:slug`	optional	Details, genres, episodes (no URLs), `minTier`, `canWatch`, `hasSource`, rating avg/count, user rating, in-watchlist
PUT `/titles/:id/rating`	user	`{stars:1..5}`
POST `/playback`	optional	`{titleId, episodeId?, clientId}` → `{sessionId, heartbeatSec, source:{url, kind:'file'|'embed'}}` after tier + screen checks; counts the view. Anonymous viewers get `sessionId: null` and only free titles
POST `/playback/:id/heartbeat`	user	Keeps the screen slot alive; re-checks entitlement
DELETE `/playback/:id`	user	Releases the slot
GET `/watchlist`	user	Titles in the list
PUT `/watchlist/:titleId`	user	Add (idempotent)
DELETE `/watchlist/:titleId`	user	Remove
GET `/progress/:titleId`	user	Progress rows for a title
PUT `/progress`	user	`{titleId, episodeId?, position, duration}`
GET `/progress`	user	Continue-watching list

Method & path	Auth	Description
GET `/plans`	–	`{plans, checkoutEnabled, provider, periodDays}`
GET `/billing/subscription`	user	Subscription (also an expired one, `active:false`), access, last 20 payments
POST `/billing/checkout`	user	`{plan}` → 201 (demo provider only; 503 `billing_disabled` otherwise; 409 if already on that plan)
POST `/billing/cancel`	user	No renewal; access until period end
POST `/billing/resume`	user	Undo cancel while the period is still running
POST `/admin/titles`	admin	Create title (+genres, `minTier`, `rightsNote`)
PUT `/admin/titles/:id`	admin	Update title
DELETE `/admin/titles/:id`	admin	Delete title (cascade)
GET `/admin/titles`	admin	All titles incl. drafts, with `videoUrl`, `rightsNote`, `sourceKind`
GET `/admin/titles/:id/episodes`	admin	Episodes with `videoUrl`, `sourceKind`
POST `/admin/titles/:id/episodes`	admin	Add episode (embed requires the title's rights note)
DELETE `/admin/episodes/:id`	admin	Remove episode
POST `/admin/subscriptions`	admin	`{email, plan, days}` → grant (provider `admin`)
POST `/admin/subscriptions/revoke`	admin	`{email}` → remove subscription and playback sessions

Removed in 1.1: `POST /titles/:id/view` (views are now counted when playback is authorised, so a client cannot inflate them without passing the checks).

Validation rules: email RFC-lite regex ≤ 254 chars; username 2–32 chars `[\w .-]`; password 8–72 chars (bcrypt limit); `clientId` 8–64 chars `[\w-]`; video URLs must be `http(s)://`; VK-host URLs must be an `https://…/video_ext.php?oid=…&id=…[&hash=…][&hd=1-5]` embed (rebuilt from validated parts); text lengths bounded; page size ≤ 50; all identifiers integers.

4. Security design

Assets: accounts, watch history, catalog integrity, availability.

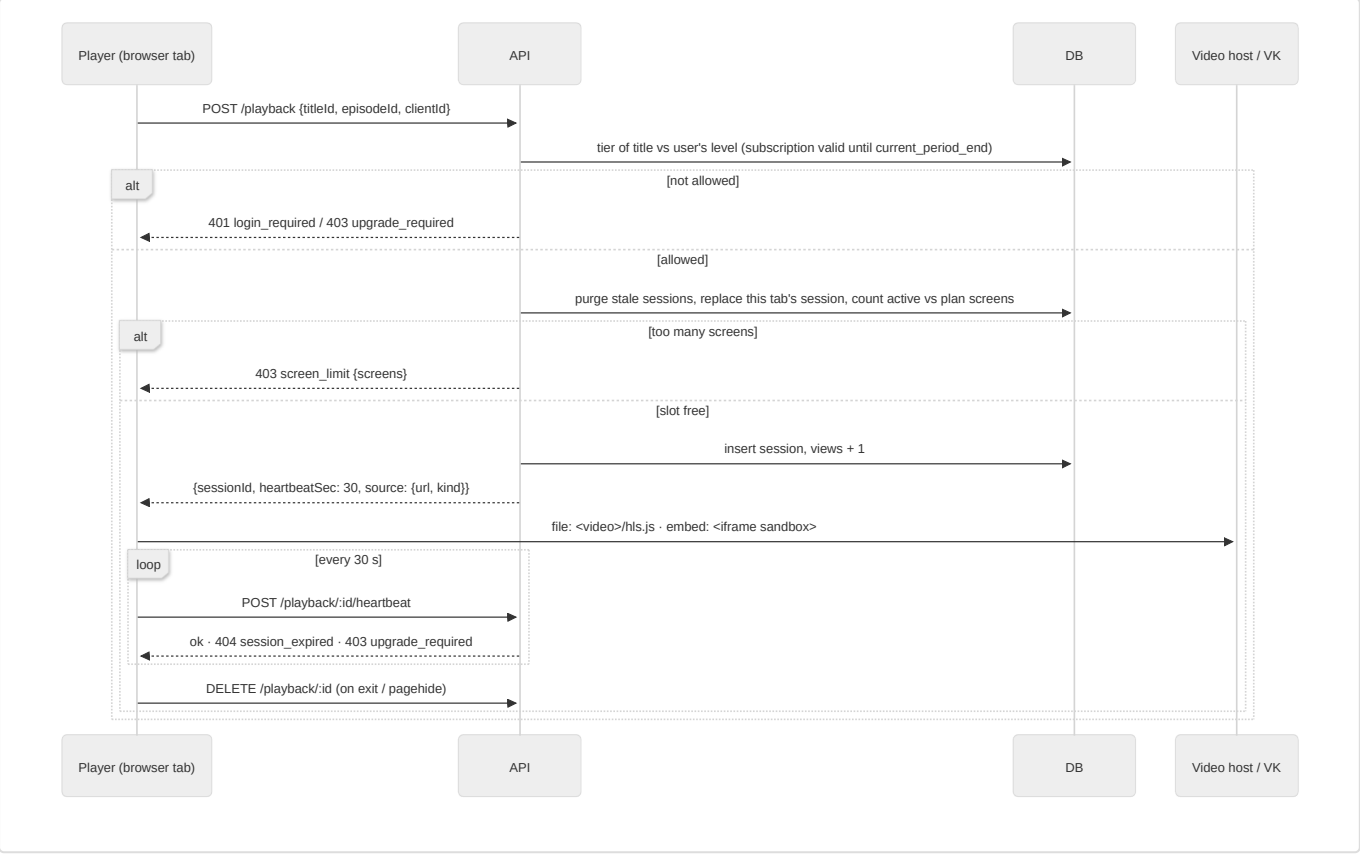
STRIDE	Threat	Control
Spoofing	Credential stuffing	bcrypt, 20 auth attempts / 15 min / IP, generic "invalid credentials"
Tampering	SQL injection	prepared statements only
Tampering	CSRF	SameSite=Strict cookie, JSON-only bodies, `Origin` check on mutating requests
Repudiation	Admin changes untracked	Request logging with user id (full audit table in roadmap)
Info disclosure	XSS stealing session	httpOnly cookie, CSP `script-src 'self'`, no `innerHTML` with data
Info disclosure	User enumeration	same response for unknown email vs bad password; register conflict is the one accepted leak (documented)
DoS	Request floods	global rate limit 300 req/min/IP, body limit 100 kB, pagination caps
Elevation	User hits admin API	`requireAdmin` middleware on `/api/admin`, role read from DB per request
Elevation	Free user watches a paid title by calling the API directly	Tier check in `POST /playback`; public payloads contain no URLs; client UI is cosmetic only
Tampering	Client inflates views	Views counted only inside an authorised `POST /playback`
Elevation	Account sharing beyond the plan	Per-user screen limit with heartbeat/expiry; own-tab replacement; entitlement re-checked on each heartbeat

STRIDE	Threat	Control
Info disclosure	Copying the media URL after authorisation	**Not preventable without signed URLs/DRM** — accepted for v1.1 (see R9, roadmap v1.4/v2.0)
Tampering	Fake payment / self-granted plan	`BILLING_PROVIDER=none` by default in production (checkout returns 503); demo provider is dev-only; admin grants are audited via `provider='admin'`, `provider_ref`
Tampering	Malicious iframe source	Embed hosts allow-listed (`vkvideo.ru`, `vk.ru`, `vk.com`), HTTPS and `/video_ext.php` only, URL rebuilt from validated parts, iframe `sandbox`, CSP `frame-src` limited to those hosts
Info disclosure	Payment data	No card data collected or stored; only plan, amount, currency, status, provider reference

Headers via helmet: CSP (`default-src 'self'; img-src 'self' data: https:; media-src 'self' blob: https:; connect-src 'self' https:; script-src 'self'; style-src 'self'; frame-src https://vkvideo.ru https://vk.ru https://vk.com; frame-ancestors 'none'`), `X-Content-Type-Options`, `Referrer-Policy`; HSTS and `upgrade-insecure-requests` are added only when `COOKIE_SECURE=true` (i.e. when served over HTTPS), because they would break a plain-HTTP run. `connect-src https:` is needed so hls.js can fetch playlists/segments from the CDN; tighten to an allow-list of the operator's hosts in production (`MEDIA_ORIGINS` env).

Secrets: `JWT_SECRET` mandatory in production (process exits otherwise). Default admin credentials are never created in production unless `ADMIN_EMAIL` and `ADMIN_PASSWORD` are provided.

5. Video delivery and playback authorisation



- **Sessions:** a slot is a row in `playback_sessions`; it expires 90 s after the last heartbeat, so crashed or sleeping clients free their slot. The per-tab `clientId` (kept in `sessionStorage`) lets a tab switch episodes without counting against itself.
- **Entitlement window:** a subscription grants access while `current_period_end` is in the future, whether `active` or `canceled`. Admin accounts are treated as Diamond.

- **File sources:** URL pointer to MP4 (progressive download, HTTP range) or HLS master playlist. If the URL ends with `.m3u8` and the browser lacks native HLS → hls.js with automatic quality switching; otherwise set `video.src`. On fatal error show a retry panel.
- **VK embeds:** an `<iframe>` to the official player with `sandbox="allow-scripts allow-same-origin allow-presentation allow-popups"` and `allow="autoplay; encrypted-media; fullscreen; picture-in-picture"`. Because the player is cross-origin, **progress saving, resume, next-episode autoplay and keyboard shortcuts do not apply** to embeds; the UI says so.
- **Progress (file sources):** saved every 10 s, on pause, on `visibilitychange` and `pagehide` via `fetch(keepalive)`.
- **Target pipeline (v1.4):** upload → FFmpeg ladder (240p–1080p, AAC, H.264) → HLS segments in object storage → CDN with short-lived signed URLs → optional DRM (v2.0).

6. UI / UX design

Information architecture

```
Home ─┬─ Browse (Free | Movies | Series | Anime, access chips, genre chips, sort)
      ├─ Search
      ├─ Title — Watch (player + episode list; login / upgrade / screen-limit states)
      ├─ Watchlist
      ├─ Plans — Checkout (demo)
      ├─ Account (subscription, cancel/resume, payment history)
      ├─ Login / Register
      └─ Admin (admin only: titles, tiers, rights notes, grant/revoke plans)
```

Visual language (v1.1): black/charcoal surfaces (`#141414`, `#1f1f1f`), signal red `#e50914` for primary actions and the wordmark (white text on it ≈ 4.8:1), lighter red `#ff4d55` for red text on dark (≥ 5:1), white/grey text; a bold uppercase **MOVIEON** wordmark (no third-party logos or names); squared 4–6 px corners; full-bleed hero with a dark gradient into the page; Helvetica/Arial-style system font stack (no external fonts); 8 px spacing scale; posters 2:3.

Tier colours: Free = charcoal, Silver `#c9ced6`, Gold `#f2c14e`, Diamond `#7fdcff` — always shown as a text label (never colour alone). Locked posters are desaturated and carry a lock icon.

Icons (1.1.1). `web/js/icons.js` exports `icon(name, {size, class})`, which builds an inline `<svg viewBox="0 0 24 24">` with `createElementNS` (the DOM helper `h()` knows the SVG namespace, so no `innerHTML`, no sprite fetch and no `style` attribute — compatible with `script-src 'self'` and `style-src 'self'`). Line icons use `stroke="currentColor"`, 2 px round caps; `play` and the small dots are solid. Parts that animate carry classes (`ic-shackle`, `ic-check` with `pathLength=1`, ...). Helpers: `TIER_ICON` (shield / crown / gem), `equalizer()` (now-playing bars), `spinner()` (player loader). Emoji and glyph characters are not used anywhere (an e2e step scans the visible text for them).

Motion (1.1.1). Principles: (1) the un-animated state is always the final, visible state and keyframes describe only the way in, so disabling animation can never hide content; (2) only compositor-friendly properties (`transform`, `opacity`, `stroke-dashoffset`, `box-shadow`) are animated; (3) JavaScript only toggles classes or sets CSS custom properties (`--i` stagger index, `--dur`, `--rx/--ry` ripple origin) through the CSSOM; (4) `@media (prefers-reduced-motion: reduce)` switches off all animation and transition, the splash and hero auto-rotation. Easing tokens: `--ease-out: cubic-bezier(.22,1,.36,1)` and an overshoot `--spring`. Implemented pieces: `splash()` in `main.js` (sessionStorage flag, closes after ≈ 1.5 s or on click / key), `route-loading` class on `<html>` driving the top progress bar, document-level pointer handler for the ripple, scroll listener (passive) for the header, `celebrate()` in `ui.js` (SVG ring/tick + 22 CSS-driven particles), countdown ring driven by one CSS animation of `--dur` seconds while a 1 s timer updates the number and an `aria-live` text.

Design tokens (CSS custom properties): `--bg, --surface, --surface-2, --border, --text, --muted, --accent, --accent-strong, --silver, --gold, --diamond, --radius, --shadow, --ease-out, --spring`. The page declares `color-scheme: dark`; there is no theme toggle.

Accessibility: semantic landmarks, skip link, visible focus rings, labelled controls, `aria-live` for toasts, `prefers-reduced-motion` honoured (all motion off, splash skipped), decorative icons are `aria-hidden`, the countdown number is mirrored in a polite live region, cards are real links and their accessible name includes the plan ("…, Gold plan" / "…, free"), tier is never conveyed by colour alone.

States designed for every list: loading skeleton, empty, error with retry. **Player states:** playing, playback problem (retry), sign-in required, upgrade required, screen limit, next-episode countdown.

7. Scalability path

1. Put the container behind a reverse proxy/CDN; cache `/vendor` and static assets (immutable, hashed in v1.1).
2. Move DB to PostgreSQL (schema uses portable SQL; swap the thin `db.js` layer).
3. Add Redis for rate-limit counters, view-count buffering and cache of `/api/home`.
4. Split video ingest/transcoding into a worker service with a queue.
5. Search: Postgres `pg_trgm`/full-text first, dedicated engine later.

8. Entitlements and billing design

Plans (single source of truth: `server/src/plans.js`; prices are placeholders in USD):

Plan	Tier unlocked	Screens	Price / 30 days
Free (no plan)	`free`	1	0
Silver	`free`, `silver`	1	$4.99
Gold	+ `gold`	2	$8.99
Diamond	+ `diamond`	4	$12.99

Rules

1. Level = 0 (free) · 1 Silver · 2 Gold · 3 Diamond; a user may watch a title when `level ≥ level(title.min_tier)`.
2. Subscription valid while `current_period_end > now`. `canceled` only means "do not renew".
3. Changing plan starts a **new 30-day period at the full price** (no proration — documented limitation). Subscribing to the plan you already have returns 409.
4. The demo provider does **not** renew automatically; when the period ends the user falls back to Free.
5. Not enforced: picture quality per plan (each title has one source), download limits, device types.

Provider boundary. `BILLING_PROVIDER=mock` (default in development) enables `POST /billing/checkout`, which writes a `subscriptions` row and a `payments` row in one transaction. `none` (default in production) disables it and the UI says payments are not enabled. A real provider replaces checkout with a redirect to the provider's hosted page and writes the same two rows from a **signed webhook** (verify signature, idempotent on `provider_ref`, handle renewal/failed payment/refund) — see `04-deployment-and-operations.md` §12.

MovieOn — Phase 4: Testing

Version 1.1.1 · 2026-10-20

1. Strategy

Level	Tool	Scope	Where	Run
Unit/API integration	Node built-in test runner (`node:test`) + real HTTP against the Express app on an in-memory SQLite DB	Every REST endpoint, validation, authorisation, tier/screen enforcement, billing lifecycle, embed validation, migrations, security headers, rate limiting	`server/test/` (`api.test.js`, `subscriptions.test.js`)	`cd server && npm test`
End-to-end (browser)	Playwright + Chromium against the real server and UI	Core user journeys, player behaviour, locked-title flow, plans → demo checkout → unlock, VK embed (stubbed host), dark red/black design, mobile layout, CSP cleanliness	`e2e/smoke.mjs`	`cd e2e && npm install && npx playwright install chromium && npm test`
Performance	autocannon (ad-hoc)	Catalog read endpoints and playback authorisation	see section 5	manual
Dependency security	`npm audit`	Production dependencies	CI job `test`	automatic
Manual / exploratory	Checklist in section 6	Real devices, real video hosts	–	before each release

Design choices: tests exercise the public HTTP contract rather than internals, so refactoring does not break them. The e2e job substitutes the third-party demo video URLs with a locally generated WebM that is served **with HTTP Range support** (needed by browsers for seeking, i.e. resume), so player logic is tested without internet access or dependence on external hosts. The VK embed host is **stubbed** (the iframe loads a placeholder page): the tests verify validation, storage, the sandboxed iframe and CSP — not VK's real player.

2. Requirements traceability

Requirement	Automated verification
FR-01 Register	API: *register validation*, *register → cookie flags → me → logout*, *duplicate email → 409* · E2E: *register signs the user in*
FR-02 Login/logout/session	API: *login: wrong password and unknown email give the same message*, *tampered cookie is treated as anonymous* · E2E: *sign out; admin signs in…*
FR-03 Home rows	API: *home: hero, rows, empty continue-watching for anonymous* · E2E: *home renders hero and rows*, *player… Continue watching*
FR-04 Filter/sort	API: *filter by type*, *filter by genre…*, *sort by name…*, *pagination covers disjoint pages*, *invalid query params → 400* · E2E: *browse anime + genre filter*
FR-05 Search	API: *search by name and synopsis…*, *LIKE wildcards…*, *SQL injection attempts…* · E2E: *search finds Orbit Station*
FR-06 Title details	API: *detail payload never exposes video URLs; series episodes are ordered*, *unknown slug → 404* · E2E: *title page: details, watchlist, rating*
FR-07 Playback	E2E: *player: plays, saves progress, resumes* (MP4/WebM path). HLS path is covered **manually** (section 6) — see limitation L1
FR-08 Resume / continue watching	API: *movie progress appears…*, *finished movie and tiny positions are not listed*, *series: episodeId is required…*, *position is clamped…* · E2E: *player: plays, saves progress, resumes*

Requirement	Automated verification
FR-09 Watchlist	API: three tests in suite *FR-09 watchlist* · E2E: *title page…*, *watchlist page lists the title*
FR-10 Ratings	API: *validation, update and aggregation* · E2E: *title page…*
FR-11 Next episode	API: *finishing an episode surfaces the next one; finishing the last drops the series* · E2E: *player: next-episode overlay and Play now*
FR-12 Admin	API: six tests in suite *FR-12 admin* (authorisation, draft lifecycle, validation, unique slugs, draft isolation, live role changes) · E2E: *sign out; admin signs in and manages catalog*
FR-13 Trending	API: *FR-13: playback start counts a view and feeds "popular" sort*
FR-14 Responsive/keyboard	E2E: *mobile viewport has no horizontal overflow* (390 px) · accessibility audit is manual (section 6)
FR-15 Shortcuts	E2E: *keyboard shortcut M toggles mute* (other shortcuts manual)
FR-16 Red/black design	E2E: *design is dark red/black regardless of OS preference; no theme toggle* (checks body `rgb(20,20,20)`, primary button `rgb(229,9,20)`, no theme control, under both OS colour schemes)
FR-17 Tiers	API: *admin: tiers and tier filter*, *list returns paginated items without video URLs*
FR-18 Plans	API: *GET /api/plans lists Silver < Gold < Diamond…*, *plan ladder: each plan unlocks its own tier and below, never above*
FR-19 Server-side enforcement	API: *anonymous: free title plays…*, *paid title → 401*, *403 carries a machine-readable code…*, *playback validates input*, *detail payload never exposes video URLs* · E2E: *gated title: server refuses playback and the player explains why* (also asserts the public JSON contains no media URL)
FR-20 Checkout / account	API: five tests in suite *checkout and subscription lifecycle* (auth + validation, subscribe → upgrade → history, cancel/resume/expiry, 503 without provider, admin grant/revoke) · E2E: *plans page → demo checkout → Gold unlocks the title*, *cancel keeps access until period end; resume restores*
FR-21 Screens	API: five tests in suite *simultaneous screens* (1 / 2 / 4 screens, own-tab replacement, release on stop, stale expiry, heartbeat 200/404/403, session ownership)
FR-22 Free catalog	API: *create/update minTier; catalog ?tier= filter…* (home *Free to watch* row asserted there) · E2E: *browse: Free filter shows only free titles*, *home renders hero and rows*
FR-23 VK embed	API: five tests in suite *VK embed sources* (page links rejected, rights note required, normalisation + `kind=embed`, malformed/non-VK hosts, episode rule, CSP `frame-src`) · E2E: admin step creating *VK Demo* (page link refused → rights note required → sandboxed iframe rendered)
FR-24 Admin tier / grant	API: *admin: tiers and tier filter*, *admin can grant and revoke subscriptions; users cannot* · E2E: admin form
FR-25 Locked-state UI	E2E: *title page…* asserts "Unlock with Gold"; *gated title…* asserts the overlay; plans/account screenshots reviewed
FR-26 Icon set	E2E: *icons: custom SVG set, no emoji or glyph icons anywhere in the UI text* (≥ 10 `svg.icon`, all `aria-hidden`, visible text contains no pictographic or arrow/star/check glyph)
FR-27 Motion	E2E: *animations: splash once per session, entrance + hover motion present* (letters drawn, splash removed and not repeated after reload, `.enter` has an animation, header `scrolled` class, Play button fades in on hover, ripple on press, no console errors) · *prefers-reduced-motion: no splash, no running animations* (no infinite animation, `animation-name: none` on entrance elements) · the checkout step asserts the celebration overlay
NFR-11 Migrations	API: *a v1.0 database is upgraded in place and keeps its data* (creates a v1 DB, reopens, checks version, data and defaults, idempotent reopen)
NFR-02 Security	API: *security headers…*, *cross-origin state-changing request is blocked*, *auth endpoints return 429 after 20 attempts*, SQL-injection and authorisation tests · E2E: *no CSP violations or script errors in the console* · CI: `npm audit`
NFR-08 Observability	API: *GET /api/health*

3. Current results (v1.1.1, 2026-10-20)

Suite	Result
API / integration	**66 passed, 0 failed** (16 suites; 42 from v1.0 adapted, 24 new)
End-to-end (Chromium headless, 1280×800 and 390×844)	**20 / 20 steps passed** (3 new in 1.1.1: icons, splash + motion, reduced motion), 0 script errors, 0 CSP violations. Expected 4xx responses (locked title, rejected VK link) show up as browser "Failed to load resource" lines and are filtered by the test
`npm audit` (server production deps, e2e)	0 vulnerabilities
Production-mode smoke (container file layout, run directly with Node)	start-up OK; refuses to start without `JWT_SECRET`; static client and vendored hls.js served; session cookie issued over HTTP when `COOKIE_SECURE=false`; CSP without HSTS/`upgrade-insecure-requests` on plain HTTP
Docker image build	**Not executed** — no Docker daemon in the build environment. The CI job `docker` builds the image and checks `/api/health`; run `docker compose up --build` once locally before release

Defects found and fixed during testing (v1.0 items 1–4, v1.1 items 5–8, v1.1.1 items 9–10):

1. Admin edit form would have blanked a movie's source URL (list endpoint did not expose it) → admin list now returns `videoUrl`; form omits an empty URL instead of sending null.
2. `upgrade-insecure-requests` and HSTS were tied to `NODE_ENV=production`, which would break a production container reached over plain HTTP (e.g. local `docker compose up`) → now tied to `COOKIE_SECURE=true`.
3. Hero text was unreadable-prone in light theme (text colour followed the theme while the artwork is always dark) → hero now always uses light text over a dark scrim.
4. Resume looked broken in e2e; root cause was the test stub lacking HTTP Range support, not the app. Stub now behaves like a CDN. (Documented because real hosts **must** support Range for resume to work.)
5. *(v1.1)* My first plan-ladder test failed because it started several playbacks in one tab for a 1-screen plan; the app was right (each new client id is a new screen). Test now uses one client id per viewer, as a real tab does.
6. *(v1.1)* Two e2e races found by repeated runs (each ≈ 1 failure in 4): the plans step counted cards before the data had loaded, and the browse step counted cards of the previous page during navigation. Both now wait for the final DOM state (`.grid:not([aria-busy])`, `.plan-diamond`). Application behaviour was correct.
7. *(v1.1)* Browse page rendered the literal text `null` under the grid when there was only one page (`replaceChildren` stringifies `null`; a v1.0 latent bug visible in the new screenshots) → rendering now goes through the null-skipping DOM helper (also applied to the admin table).
8. *(v1.1)* Header showed the **Free** badge for admins (who have Diamond-level access) → shows **Admin**. Nav links were clipped at 1280 px → no-shrink links and narrower search field.
9. *(v1.1.1)* Found in code review before the first run: the plan-activation overlay reused the class `tier-gold`, which also carries the badge's solid gold background, so the whole screen would have turned gold. Renamed to `cel-gold` / `cel-silver` / `cel-diamond`.
10. *(v1.1.1)* The first run of the new "no emoji" step failed on `©` in the footer (Unicode classifies it as pictographic). That was a test bug, not an application defect; the check now ignores `©`. The title-page screenshot also showed a half-filled star row because the pointer hovered a star after the label width changed; the test now parks the pointer first.

4. Test cases — representative detail

ID	Case	Steps	Expected
TC-AUTH-04	Enumeration resistance	Login with a registered email + wrong password; login with an unknown email	Both: 401, identical message
TC-CSRF-01	Cross-origin write	`POST /api/auth/login` with `Origin: http://evil.example`	403
TC-CAT-07	Wildcard literal	`GET /api/titles?q=%`	0 results (not "everything")
TC-PRG-05	Series completion	Finish episode 1 of a 4-episode series	Continue-watching shows episode 2 at 0:00; finishing episode 4 removes the series from the list
TC-ADM-03	Draft isolation	Admin creates draft; anonymous requests it	List excludes it; detail 404; watchlist/rating/view calls 404
TC-ADM-06	Role revocation	Promote then demote a user in the DB	Next request reflects the change (role read per request, not baked into the token)
TC-RATE-01	Brute force	22 failed logins from one client	429 after the 20th
TC-ENT-01	Direct API bypass	Anonymous `POST /api/playback` for a Gold title	401 `login_required`, no URL in the body
TC-ENT-02	Upgrade ladder	Free / Silver / Gold / Diamond users request one title of each tier	Free: 200,403,403,403 · Silver: 200,200,403,403 · Gold: 200,200,200,403 · Diamond: all 200
TC-ENT-03	Expiry	Set `current_period_end` in the past	`/auth/me` level 0; playback 403; heartbeat of a running session 403 and slot removed
TC-SCR-01	Screen limit	Silver user starts playback in tab A, then tab B	Tab B 403 `screen_limit`; after A stops (or 90 s without heartbeat) B succeeds
TC-SCR-02	Tab reuse	Same tab starts playback twice	Second call replaces the first session, no limit hit
TC-BIL-01	Demo checkout	Subscribe Silver → Gold	201 each; payments list newest first (Gold 899¢, Silver 499¢); same plan again → 409
TC-BIL-02	No provider	Boot with `billingProvider:'none'`; checkout	503 `billing_disabled`; `/plans` reports `checkoutEnabled:false`
TC-VK-01	VK page link	Admin posts `https://vk.com/video-…`	400 with instructions to use the official embed link
TC-VK-02	Rights note	Valid embed link without rights note	400; with note → stored without `autoplay`/unknown params, playback `kind:'embed'`
TC-MIG-01	Upgrade in place	Create a v1.0 DB (user_version 0) with one title, open with v1.1	version 2, title kept as `free`, tables present, reopening is a no-op

5. Performance measurement

Tool: autocannon, 50 concurrent connections, 8 s per endpoint, server and load generator on the **same** 2-vCPU sandbox (so numbers are conservative for the server but not a substitute for a real environment), rate limiting disabled for the run, SQLite file DB with the 13-title demo catalog. Re-measured for v1.1.

Endpoint	Throughput (req/s)	p50 latency	p97.5 latency	Errors / non-2xx	v1.0 for reference
`GET /api/titles?limit=24`	≈ 1 080	42 ms	72 ms	0	≈ 1 090
`GET /api/titles?q=orbit`	≈ 1 250	36 ms	67 ms	0	≈ 1 430
`GET /api/titles?tier=free`	≈ 1 265	37 ms	61 ms	0	–

Endpoint	Throughput (req/s)	p50 latency	p97.5 latency	Errors / non-2xx	v1.0 for reference
`GET /api/titles/harbor-lights`	≈ 1 550	29 ms	53 ms	0	≈ 1 390
`GET /api/home`	≈ 375	127 ms	192 ms	0	≈ 510
`POST /api/playback` (anonymous, free title; includes the view-counter write)	≈ 1 300	35 ms	68 ms	0	–

Reading: NFR-01 (p95 < 200 ms at 100 req/s) is still met **for this catalog size**, but `/api/home` is the slowest endpoint and got ~25 % slower in v1.1 (one more list query for the *Free to watch* row). It is the first candidate for caching (roadmap v1.3). The signed-in `POST /api/playback` path (session table write inside `BEGIN IMMEDIATE`) was **not** load-tested; SQLite serialises writers, so it is the first write path to watch as concurrent viewers grow. Real-world numbers depend on catalog size (LIKE search is a full scan; move to full-text/trigram beyond a few thousand titles) and hardware. Re-run before launch with production-like data.

6. Manual test checklist (before each release)

Playback (cannot be automated reliably against third-party hosts)

- ☐ MP4 title plays on Chrome, Firefox, Edge, Safari (macOS) and iOS Safari.
- ☐ HLS (`.m3u8`) title plays: hls.js path on Chrome/Firefox/Edge, native path on Safari/iOS; quality switches on throttled network.
- ☐ Seek works on a real host (confirms the host serves HTTP Range).
- ☐ Source failure shows the "Playback problem" panel; Retry resumes near the previous position.
- ☐ Shortcuts: Space, ←/→, F, M, N (when a next episode exists).
- ☐ **Real VK embed** (needs an owned/licensed video with embedding enabled): plays in VK's player inside the sandboxed iframe on desktop and mobile; a removed/private video shows VK's own error, not a broken page.

Subscriptions (manual, with the real provider once integrated)

- ☐ Checkout → webhook → access within seconds; failed payment and refund remove access at the right time.
- ☐ Two devices on a Silver account: second is refused; closing the first frees the slot within ~90 s.
- ☐ Locked states read correctly on phone width (badges, Unlock button, Plans, Account).

Accessibility (target WCAG 2.1 AA)

- ☐ Full keyboard walkthrough: skip link, nav, carousel dots, filters, forms, admin table.
- ☐ Screen reader pass (NVDA/VoiceOver): landmarks, form labels, toasts announced, chips expose pressed state.
- ☐ Contrast spot-check (red text on `#141414`, tier badges, muted text); focus ring visible everywhere.
- ☐ 200 % zoom and `prefers-reduced-motion` (no auto-rotating hero, no shimmer, no splash, spinner fades instead of rotating).
- ☐ Motion on a mid-range Android phone and in Safari/Firefox: no jank on Home and Plans, splash not annoying, celebration readable.

Security

- ☐ Production deployment serves HTTPS, `COOKIE_SECURE=true`, HSTS present, `JWT_SECRET` unique.
- ☐ Default dev admin does **not** exist in production (`ADMIN_EMAIL`/`ADMIN_PASSWORD` set explicitly).
- ☐ `MEDIA_ORIGINS` restricted to the operator's video hosts.

- [ ] External scan (e.g. OWASP ZAP baseline) shows no high findings.

Content & legal

- [ ] Every catalog title has documented distribution rights; demo placeholder titles removed or replaced.
- [ ] Takedown procedure rehearsed (unpublish → verify 404 for anonymous users).

7. Known limitations of the test suite

- **L1** HLS playback is not covered by automation (needs a real, reachable HLS source and codec support that headless Chromium lacks for some profiles).
- **L2** Only Chromium is exercised by e2e; Firefox/WebKit are manual.
- **L3** No load test beyond the ad-hoc read-endpoint measurement; no soak test for SQLite WAL growth.
- **L4** Accessibility is checked manually; no automated axe run yet (candidate for v1.1).
- **L5** Docker image build is verified only by the CI job, not in this environment.
- **L6** Withholding video URLs stops casual bypass but not a paying user who copies the URL once issued (test TC-ENT-* prove the checks, not URL secrecy). Signed URLs/DRM are roadmap.
- **L7** Billing is tested against the **demo** provider only. No real payment, webhook signature, renewal, refund or tax behaviour has been exercised.
- **L8** The VK embed is tested against a stub host; the exact `video_ext.php` parameters follow VK's documented embed format but were **not** verified against a live VK video here.
- **L10** *(1.1.1)* Motion is verified structurally (elements, computed styles, running animations, screenshots with animations frozen), **not** visually frame by frame, and only in Chromium. Smoothness on low-end phones, Firefox/Safari rendering of `backdrop-filter`, `:has()` (star hover preview) and `color-mix()`, and frame-rate / battery impact were not measured; see the manual list.
- **L9** Screen-limit tests exercise the API with synthetic client ids; multi-device behaviour on real networks (sleep/wake, flaky connections) is manual.

8. Exit criteria for v1.1

- All automated suites green in CI (API, e2e, docker build + health check).
- No open defect of severity High/Critical; known limitations above accepted by the product owner.
- Manual checklist (section 6) signed off on at least Chrome (desktop + Android) and Safari (macOS + iOS).
- Content-rights confirmation in place for every published title, **including a rights note for every VK embed**.
- Before charging real money: a real payment provider integrated, its webhook path tested, `BILLING_PROVIDER` not `mock` in production, prices and plan terms reviewed by the owner (and legal/tax advice obtained).

MovieOn — Phases 5–6: Deployment & Maintenance

Version 1.1.1 · 2026-10-20

> **Changes in 1.1.1:** none for operators — client-only release (icons, animations); same configuration, image and database schema.

> **Changes in 1.1:** `BILLING_PROVIDER` setting, VK frame allow-list in the CSP, automatic schema upgrade (migrations), subscription/payment operations (§12), content-compliance rules for embeds.

1. Environments

Env	Purpose	Data	Notes
Local dev	Daily work	Demo seed, SQLite file in `server/data/`	`npm run dev`; dev admin `admin@movieon.local` / `admin12345` is created **only** when `NODE_ENV` is not `production` and no admin env vars are set
CI	Automated checks	In-memory SQLite	GitHub Actions: API tests + audit → browser e2e → Docker build and health check
Staging	Pre-release verification, manual checklist	Copy of production catalog (no user data)	Same container, `COOKIE_SECURE=true` behind TLS
Production	Live	Real catalog, `SEED_DEMO=false`	Single container + persistent volume behind a TLS-terminating reverse proxy/CDN

2. Configuration

Variable	Default	Required in prod	Meaning
`NODE_ENV`	–	yes (`production`)	Enables production defaults
`PORT`	3000	no	Listen port
`DB_PATH`	`./data/movieon.db` (`/data/movieon.db` in the image)	no	SQLite file; mount a volume
`JWT_SECRET`	random per boot in dev	**yes** — process exits if missing	Session signing key; rotating it logs everyone out
`ADMIN_EMAIL`, `ADMIN_PASSWORD`	dev admin only outside production	**yes** (first start)	Applied only when that account does not exist yet. v1.0 has no password-change or role UI — use the admin CLI (section 6) to reset a password or promote a user
`COOKIE_SECURE`	`true` in production	set `true` behind HTTPS	Secure cookie flag; also enables HSTS and CSP `upgrade-insecure-requests`. Set `false` only for plain-HTTP local runs
`SEED_DEMO`	`true`	set `false`	Insert demo catalog when the DB is empty
`MEDIA_ORIGINS`	any `https:`	recommended	Comma-separated video/CDN origins allowed by CSP (`media-src`, `connect-src`)
`TRUST_PROXY`	off	yes behind a proxy	Number of proxies, so rate limiting uses real client IPs

Variable	Default	Required in prod	Meaning
`BILLING_PROVIDER`	`mock` in development, `none` in production	**keep `none` until a real provider exists**	`mock` = demo checkout that records a payment without charging anyone or collecting card data. `none` = checkout disabled (HTTP 503; the UI says payments are not enabled); plans can still be granted by an admin. The server logs a warning if `mock` is used in production
`WEB_DIR`	`../web`	no	Static client directory

3. Running

Without Docker (Node 22+)

```
cd server
npm ci
npm start                       # http://localhost:3000
npm test                        # 66 API tests
```

With Docker Compose

```
cp .env.example .env            # set JWT_SECRET, ADMIN_EMAIL, ADMIN_PASSWORD
docker compose up --build -d
curl http://localhost:3000/api/health
```

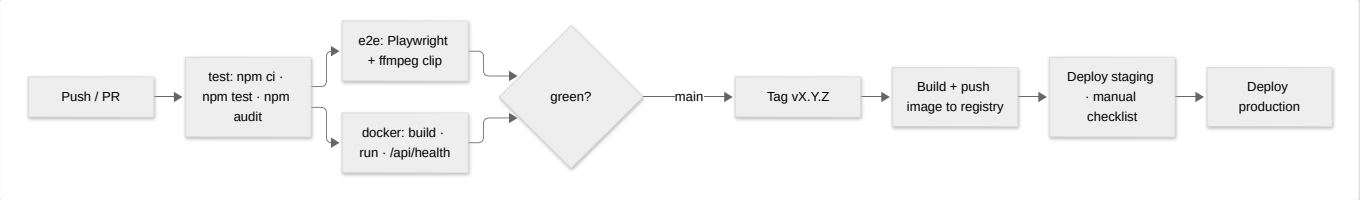
The image is multi-stage (build tools only in the dependency stage), runs as the unprivileged `node` user, stores data in the `/data` volume, and has a `HEALTHCHECK` on `/api/health`.

Production topology (recommended)

```
Internet ─► CDN / reverse proxy (TLS, HTTP/2, gzip/brotli, WAF)
              ├─► MovieOn container  (/, /api, /vendor)
              └─► Video origin / object storage (MP4, HLS) — served directly to browsers, with HTTP Range
```

Reverse proxy checklist: terminate TLS; forward `X-Forwarded-For`/`-Proto`; set `TRUST_PROXY=1`; compress text assets; cache `/vendor/*` and static files; **do not** cache `/api/*`.

4. CI/CD pipeline



The repository contains the CI stages (`.github/workflows/ci.yml`). Registry push and deployment are environment-specific and intentionally left to the operator (they need credentials/targets that do not belong in the repo).

5. Release process

1. Branch from `main`; one PR per change; CI must be green; at least one reviewer.
2. Versioning: SemVer. Breaking API/DB changes bump the major version.
3. Release candidate → staging → manual checklist (`03-testing.md` §6) → production.
4. **Rollback:** redeploy the previous image tag. Schema changes must be backward compatible for one release (expand → migrate → contract) so a rollback never needs a DB restore.
5. Changelog entry per release; known limitations updated.

6. Database operations

- SQLite runs in WAL mode; the database consists of `movieon.db`, `-wal`, `-shm`.
- **Backup:** use SQLite's online backup, not a raw copy of a live file:

  ```
  docker compose exec movieon node -e "const D=require('better-sqlite3');new
  D('/data/movieon.db').backup('/data/backup-'+Date.now()+'.db').then(()=>console.log('ok'))"
  ```

  Schedule daily, keep 14 days, copy off-host, **test a restore quarterly**.
- **Admin accounts (v1.0):** create, promote or reset a password with the CLI (new users otherwise get role `user`):

  ```
  docker compose exec movieon node src/admin-cli.js admin@example.com 'a-long-new-passphrase'
  # without Docker: cd server && npm run admin -- admin@example.com 'a-long-new-passphrase'
  ```

- **Migrations (implemented in 1.1):** `server/src/db.js` keeps an ordered list (V1 base schema, V2 tiers/subscriptions/payments/playback sessions). On start the server reads `PRAGMA user_version` and applies missing migrations, each in a transaction. A v1.0 database upgrades automatically on first start of v1.1 (existing titles become `free`). **Take a backup before upgrading** (command above). Rolling back to a v1.0 image after the upgrade is *not* supported by the schema bump — restore the pre-upgrade backup instead. Add future changes as new entries (V3, ...); never edit an applied one.
- **Subscription data:** `subscriptions` (one row per user), `payments` (append-only history), `playback_sessions` (ephemeral, rows older than 90 s are purged lazily). Payment history must be retained according to local accounting/tax rules — confirm the retention period with an accountant before deleting any `payments` rows.
- **Scale trigger:** move to PostgreSQL when sustained write contention appears (progress writes arrive every 10 s per active viewer — roughly 100 writes/s at 1 000 concurrent viewers) or when running more than one app instance.

7. Monitoring & alerting

Signal	Source	Alert
Availability	`GET /api/health` from an external prober (1 min)	3 consecutive failures → page
Error rate	JSON request logs (`s` ≥ 500)	> 1 % over 5 min
Latency	Request logs (`ms`)	p95 > 500 ms over 10 min
Auth abuse	Count of 401/429 on `/api/auth/*`	Spike vs baseline
Disk	Volume usage	> 80 %
Container health	Docker `HEALTHCHECK`	Restart loop
Video availability	Synthetic fetch of a sample of media URLs (HEAD with Range); VK embeds are checked manually	Any title unreachable for > 15 min
Playback authorisation	Request logs: share of `POST /api/playback` returning 403 (`screen_limit` / `upgrade_required`)	Sudden spike (bug or sharing wave)
Billing	Provider webhook failures, `payments.status != 'succeeded'` (once a real provider exists)	Any failed webhook for > 15 min

Logs are one JSON object per request on stdout: `{t, m, p, s, ms, u}` (time, method, path without query string, status, duration, user id). Query strings and bodies are deliberately **not** logged (they can contain search terms or credentials). Ship stdout to the platform's log service and set a retention period consistent with the privacy policy.

8. Security operations

- Rotate `JWT_SECRET` at least annually and after any suspected leak (forces re-login).
- `npm audit` runs in CI; also enable Dependabot/Renovate; patch High/Critical within 7 days.
- Rebuild the image monthly to pick up base-image fixes.
- Admin accounts: use unique long passwords; review the list of admins quarterly (v1.1 adds an audit log and 2FA).
- Incident response: contain (unpublish content or take the service offline) → preserve logs → rotate secrets → notify affected users where required by law → post-mortem within 5 working days.

9. Content compliance & takedown

1. Maintain a **rights register** outside the app: title, licensor, territories, start/end dates, contract reference.
2. Publish a contact address for rights holders and a notice-and-takedown policy (DMCA-style and local-law equivalents).
3. On a valid notice: Admin → *Unpublish* (immediate, reversible) or *Delete*; confirm the title returns 404 to anonymous users; record the action and time in the register.
4. Remove titles at licence expiry; geo-restriction support (v1.3) is required before signing territory-limited licences.
5. Do not add sources you are not licensed for. The player will play any URL an admin enters, so this is an organisational control, not a technical one.
6. **VK embeds.** Only add a VK video if the operator owns it or has written permission to show it; paste the official embed link (VK page links are rejected) and fill the **Rights note** (it is mandatory and visible to admins). Record the same item in the rights register. Do not use videos uploaded by third parties without a licence merely because they are public on VK. If VK removes a video or disables embedding, the title shows VK's error — unpublish it.
7. **Tier assignment** is a commercial decision recorded with the licence (e.g. which titles are free). Changing a title's tier takes effect immediately for new playback starts; running sessions are re-checked at their next heartbeat (≤ 30 s), so raising a title's tier also stops viewers who no longer qualify (the player is replaced by the upgrade message).

10. Maintenance plan

Cadence	Activity
Daily	Backup job result, health/alerts review
Weekly	Dependency updates, review error logs, check media-URL probe results
Monthly	Base-image rebuild, access review, capacity check (DB size, video egress cost)
Quarterly	Restore drill, accessibility re-check, penetration-test baseline, roadmap review
Annually	Secret rotation, licence/rights audit, DR exercise

Support model: issues triaged by severity — S1 (site down / data loss) 1 h response; S2 (core feature broken) 1 working day; S3 (cosmetic/minor) next sprint.

11. Roadmap and known gaps (v1.1 → next)

Gap	Impact	Planned
No real payment provider: demo checkout only, no automatic renewal, proration, invoices, tax, refunds, dunning	Cannot charge customers	v1.2 (see §12)

Gap	Impact	Planned
URL of an authorised video can be copied and shared	Paid content leakage	v1.4 signed expiring URLs, v2.0 DRM
Plans differ by catalog and screens only; no resolution tiers	Marketing copy must not promise quality tiers	v1.4 with a transcoding ladder
VK embeds: no progress/resume, no next-episode autoplay, availability controlled by VK	Weaker UX for embedded titles	accepted; prefer self-hosted licensed files
No password reset / email verification / password change	Account recovery is manual	v1.2
No subtitles / audio tracks	Accessibility and localisation	v1.2 (WebVTT)
No user data export/delete	Privacy-law readiness	v1.2
No audit log of admin actions (grants/revokes are only labelled in `subscriptions.provider_ref`)	Accountability	v1.2
Single SQLite instance, no cache; `/api/home` is the slowest read	Scale ceiling	v1.3 (PostgreSQL, Redis cache for `/api/home`)
No parental controls (labels only)	Child safety	v1.3
No upload/transcoding; videos referenced by URL	Operator must host video elsewhere	v1.4
No geo-restrictions	Needed for territory-limited licences	v1.4
UI strings inline (English only)	Localisation	v1.2 (ru, uz)

12. Payments and subscriptions — operating notes

Today (v1.1). `BILLING_PROVIDER=mock` is for demos, staging and tests only. Do **not** enable it on a public site: anyone could "buy" a plan for free. In production leave `none` and use **Admin → Grant a plan** (or `POST /api/admin/subscriptions`) for support cases, partners and pilot users; plans granted this way are labelled `provider='admin'` and the account page does not offer a cancel button for them (support revokes instead).

Integrating a real provider (checklist, not implemented).

1. Choose a provider that offers a **hosted checkout** and **signed webhooks** so card data never reaches this server (candidates to evaluate: Stripe for international customers; regional providers such as Payme/Click for Uzbekistan — availability, fees, legal requirements and API fit are *not* verified here). Settle currency (prices are USD placeholders) and tax/VAT with an accountant.
2. Replace the body of `POST /billing/checkout` with creation of a provider checkout session and a redirect; keep `GET /plans` and the UI.
3. Add a `POST /billing/webhook/<provider>` route that verifies the signature, is idempotent on `provider_ref`, and writes the same rows: upsert `subscriptions` (`plan`, `current_period_end` from the provider's period) and insert `payments`. Handle: first payment, renewal (extend the period), failed payment (grace period, then let the period lapse), cancellation, refund/chargeback (remove access).
4. Set `BILLING_PROVIDER=<provider>`; allow `checkoutEnabled` only for it. Remove/disable the `mock` branch in production builds.
5. Add monitoring for webhook failures (§7) and a reconciliation report: provider payments vs `payments` rows.
6. Publish pricing, renewal, cancellation and refund terms; obtain legal review for consumer-protection requirements in the target countries.

Support playbook.

- "*I paid but I'm still Free*" → check `GET /billing/subscription` data for the user (DB row), provider dashboard, webhook log; if confirmed paid, grant the plan for the paid period and fix the root cause.

- "*I can't watch on my second device*" → expected with 1 screen; the slot frees when the other device stops (≤ 90 s after the last heartbeat) or the user upgrades.
- *Refund / cancel* → cancel stops renewal only; access runs to period end unless support revokes (`POST /admin/subscriptions/revoke`).